ใบอนุมัติโครงงานปริญญานิพนธ์
สาขาวิชาวิทยาการคอมพิวเตอร์ ภาควิชาคอมพิวเตอร์
คณะวิทยาศาสตร์ มหาวิทยาลัยศิลปากร

ชื่อปริญญานิพนธ์ การพัฒนาระบบแจ้งเตือนการเกิดเพลิงไหม้ด้วยอินเตอร์เน็ตของสรรพสิ่ง
Development of a fire alarm system with the Internet of Things

ผู้จัดทำ นายปรัชญา สุวรรณสุข
นางสาวสุพิชชา เมธากิจภาคิน

ปีการศึกษา 2566

โครงงานปริญญานิพนธ์นี้ได้รับการอนุมัติให้เป็นส่วนหนึ่งของการศึกษาตาม
หลักสูตรวิทยาศาสตรบัณฑิต สาขาวิชาวิทยาการคอมพิวเตอร์
ภาควิชาคอมพิวเตอร์ คณะวิทยาศาสตร์ มหาวิทยาลัยศิลปากร

..... ประธานกรรมการสอบ
(ผู้ช่วยศาสตราจารย์ ดร.ณัฐโชติ พรหมฤทธิ์)

..... กรรมการสอบ
(ผู้ช่วยศาสตราจารย์ ดร. คทา ประดิษฐวงศ์)

..... อาจารย์ที่ปรึกษาและกรรมการสอบ
(อาจารย์อภิเษก หงส์วิทยากร)

การพัฒนาระบบแจ้งเตือนการเกิดเพลิงไหม้ด้วยอินเตอร์เน็ตของสรรพสิ่ง

Development of a fire alarm system with the Internet of Things

ปรัชญา สุวรรณสุข

สุพิชชา เมธากิจภาคิน

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาในหลักสูตรวิทยาศาสตรบัณฑิต
สาขาวิชาวิทยาการคอมพิวเตอร์ ภาควิชาคอมพิวเตอร์
คณะวิทยาศาสตร์ มหาวิทยาลัยศิลปากร
ปีการศึกษา 2566

ชื่อปริญญานิพนธ์ การพัฒนาระบบแจ้งเตือนการเกิดเพลิงไหม้ด้วยอินเตอร์เน็ตของสรรพสิ่ง
Development of a fire alarm system with the Internet of Things

ผู้จัดทำ นายปรัชญา สุวรรณสุข, นางสาวสุพิชชา เมธากิจภาคิน,
อาจารย์ที่ปรึกษา อาจารย์อภิเษก หงษ์วิทยากร, รองศาสตราจารย์ ดร.ปานใจ ธารทัศนวงศ์

ชื่อปริญญา วิทยาศาสตรบัณฑิต (วิทยาการคอมพิวเตอร์)
ปีการศึกษา 2566

บทคัดย่อ

บทความนี้นำเสนอรายละเอียดเกี่ยวกับการตรวจสอบอุณหภูมิของสายไฟและการแจ้งเตือนผ่านแอปพลิเคชันไลน์ รวมถึงการจัดการตัวอุปกรณ์และการดูประวัติการเกิดเหตุย้อนหลัง โดยอุปกรณ์ที่ใช้ประกอบด้วยเซนเซอร์ AMG8833 ที่เชื่อมต่อกับโมดูล ESP8266 และมีโมดูล ESP32-CAM ที่ใช้ในการจับภาพเหตุการณ์ โดยใช้การวิเคราะห์อุณหภูมิที่ได้จากเซนเซอร์ AMG8833 ในรูปแบบ Regression และ Machine Learning ด้วย Random Forest ในการสร้างโมเดล ซึ่งขั้นตอนการเตรียมข้อมูลได้ทำการจำลองสถานการณ์การใช้เครื่องใช้ไฟฟ้าที่กินกำลังเกินที่สายไฟจะรับได้ ในกรณีนี้จะกำหนดข้อมูลสำหรับการฝึกฝน (Train Set) เป็น 80% ของข้อมูลทั้งหมด และข้อมูลสำหรับการทดสอบโมเดล (Test Set) เป็น 20% ของข้อมูลทั้งหมด โดยกำหนดพารามิเตอร์คือ จำนวนต้นไม้ใน Random Forest เป็น 100 ต้นและ Random state เป็น 100 ซึ่งผลการประเมินประสิทธิภาพที่ได้คือ ค่า Mean Absolute Error (MAE), ค่า Mean Squared Error (MSE), ค่า Root Mean Squared Error (RMSE), ค่า R-squared (R2) อยู่ที่ 0.011, 0.003, 0.059 และ 0.99 ตามลำดับ

Keyword : AMG8833 ,ESP8266 ,ESP32-CAM ,Regression ,Machine Learning ,Random Forest ,Random state, Mean Absolute Error, Mean Squared Error, Root Mean Squared Error, R-squared

กิตติกรรมประกาศ

โครงงานปริญญานิพนธ์นี้เป็นการพัฒนาระบบแจ้งเตือนการเกิดเพลิงไหม้ด้วยอินเตอร์เน็ตของสรรพสิ่ง ซึ่งสามารถสำเร็จลุล่วงไปได้ด้วยดี เนื่องจากได้รับการสนับสนุนทางด้านความรู้จากบุคคลที่ผู้วิจัยทั้งสองจะกล่าวถึงบุคคลดังต่อไปนี้ ขอขอบพระคุณ อาจารย์อภิเษก หงษ์วิทยากร อาจารย์ที่ปรึกษาโครงงาน รองศาสตราจารย์ ดร.ปานใจ ธารทัศนวงศ์ อาจารย์ที่ปรึกษาร่วมโครงงาน ผู้ช่วยศาสตราจารย์ ดร.สมเจตน์ ภัทรพานิชชัย อาจารย์ภาคฟิสิกส์ นายนพรัตน์ พวงยิ้มน้อย นักวิทยาศาสตร์ชำนาญการ และนายยุทธศักดิ์ บุญรอด นักวิทยาศาสตร์ ผู้ให้ความรู้ทางด้านฟิสิกส์เกี่ยวกับระบบไฟฟ้า และความช่วยเหลือในขั้นตอนการทดลอง รวมถึง อาจารย์ ดร.สัจจาภรณ์ ไวจรรยา ผู้ช่วยศาสตราจารย์ ดร.ณัฐโชติ พรหมฤทธิ์ และ ผู้ช่วยศาสตราจารย์ ดร.คทา ประดิษฐวงศ์ ผู้ให้คำปรึกษา ให้ความรู้ และให้คำแนะนำในการศึกษาค้นคว้าตั้งแต่เริ่มต้นจนจบปริญญานิพนธ์ฉบับนี้สำเร็จผลได้ด้วยดี ขอกราบขอบพระคุณมหาวิทยาลัยศิลปากร คณะวิทยาศาสตร์ ภาควิชาคอมพิวเตอร์ที่ให้ความรู้และสถานที่ในการจัดทำโครงงานปริญญานิพนธ์นี้

สุดท้ายนี้ขอกราบขอบพระคุณบิดา มารดา และครอบครัวที่ให้กำลังใจในทุกเรื่อง ขอบคุณเพื่อนทุกคนที่มีส่วนเกี่ยวข้องในการทำโครงงาน และขอบคุณผู้จัดทำโครงงานสำหรับความตั้งใจและความพยายามที่ทำให้โครงงานสมบูรณ์และสำเร็จตามที่ได้คาดหวังไว้รวมถึงทุกคนที่มีส่วนร่วมในการช่วยเหลือให้คำแนะนำแก่ผู้วิจัยทั้งสอง จึงขอกราบขอบพระคุณเป็นอย่างสูง ณ โอกาสนี้

ปรัชญา สุวรรณสุข
สุพิชชา เมธากิจภาคิน

สารบัญ

หน้า

บทคัดย่อ ..... ก
กิตติกรรมประกาศ ..... ข
สารบัญ ..... ค
สารบัญตาราง ..... ฉ
สารบัญรูป ..... ช
บทที่ 1 บทนำ 1
1.1 ที่มาและความสำคัญของปัญหา 1
1.2 วัตถุประสงค์ 1
1.3 ลักษณะและขอบเขต 1
1.4 อุปกรณ์และเครื่องมือที่ใช้ 2
1.4.1 ฮาร์ดแวร์ 2
1.4.2 ซอฟต์แวร์ 2
1.5 ผลที่คาดว่าจะได้รับ 2
1.6 ภาระหน้าที่ความรับผิดชอบ 3
บทที่ 2 ผลงานที่เกี่ยวข้อง 4
2.1 ชื่อวิจัย : เครื่องแจ้งเตือนและควบคุมอัคคีภัยผ่าน LINE Notify [1] 4
2.2 งานวิจัย : การพัฒนาระบบตรวจวัดอุณหภูมิร่างกายแจ้งเตือนและภาพถ่ายแอพพลิเคชั่นไลน์ [2] 5
2.3 งานวิจัย : ระบบคัดกรองผู้ที่มีอาการป่วยในที่สาธารณะ [3] 6
บทที่ 3 ทฤษฎีและความรู้ที่เกี่ยวข้อง 8
3.1 NodeMCU ESP8266 V3 [4] 8
3.2 ESP32 – CAM [5] 9
3.3 เซนเซอร์ AMG8833 [6] 10
3.4 Arduino IDE [7] 10
3.5 MQTT [8] 11
3.6 Node JS [9] 12
3.7 แอปพลิเคชันไลน์โนติฟาย (LINE Notify) [10] 12
3.8 React [11] 12
3.9 Firebase [12] 13
3.10 โมเดล Random Forest Regression [13] 14
3.11 LINE Front-end Framework (LIFF) And LINE Login [14] 15
บทที่ 4 ขั้นตอนและแผนการดำเนินงาน 16
4.1 ขั้นตอนการดำเนินงาน 16
4.1.1 คิดหัวข้อและเสนอหัวข้อโครงงาน 16
4.1.2 ศึกษารวบรวมข้อมูลและกำหนดขอบเขตโครงงาน 16

4.1.3 ศึกษางานที่เกี่ยวข้องกับโครงงาน 16
4.1.4 ศึกษาความรู้และทฤษฎีที่เกี่ยวข้อง 16
4.1.5 ศึกษาการนำข้อมูลมาใช้ในงานซอฟต์แวร์ 16
4.1.6 วิเคราะห์และออกแบบระบบ 16
4.1.7 พัฒนาระบบ 16
4.1.8 ทดสอบและปรับปรุงการทำงานของระบบ 16
4.1.9 จัดทำเอกสารประกอบโครงงาน 16
4.2 แผนการดำเนินงาน 17
บทที่ 5 การวิเคราะห์และออกแบบระบบ 18
5.1 โครงสร้างการทำงานของระบบ 18
5.2 แผนผังการทำงานโดยรวมของระบบ (Flow Chart Diagram) 20
5.2.1 เว็บไซต์ของระบบ 20
5.2.2 การทำงานของอุปกรณ์และการแจ้งเตือนไลน์ 22
5.2.3 การทำงานของโมเดล (Random Forest) 24
5.3 รูปแบบเว็บไซต์ 25
5.3.1 เว็บไซต์ในส่วนของผู้ใช้ 25
5.3.2 เว็บไซต์ในส่วนของผู้ดูแลระบบ 30
5.4 การทำการทดลองเก็บข้อมูลสำหรับการทดลองโมเดล (Random Forest) 32
5.4.1 ขั้นตอนการทดลองเก็บข้อมูล 32
5.4.2 ค่าอุณหภูมิที่ได้จากการทดลองเก็บข้อมูล 33
5.5 การเทรนโมเดลด้วยข้อมูลที่ได้จากการทดลอง 35
5.6 การใช้งาน ESP32-CAM 36
บทที่ 6 ผลการดำเนินงาน ข้อจำกัด และข้อเสนอแนะ 38
6.1 ผลการดำเนินงาน 38
6.1.1 การทดสอบประสิทธิภาพของการทำนาย 38
6.1.2 การประเมิณผลการทดลอง 39
6.2 ข้อจำกัด 41
6.3 ข้อเสนอแนะ 41
บรรณานุกรม 42
ภาคผนวก ก คู่มือการติดตั้งระบบ 43
ก.1 การเซ็ทระบบ Arduino IDE 43
ก.1.1 การเพิ่มโมดูล 43
ก.1.2 ไลบรารีโมดูล ESP8266 กับ AMG8833 43
ก.1.3 ไลบรารีโมดูล ESP32-CAM 44
ก.2 การติดตั้งไลบรารีที่ใช้ใน vs code 44
ก.2.1 ไลบรารีเซิร์ฟเวอร์ 44
ก.2.2 ไลบรารีเว็บผู้ใช้ 45

ก.2.3 ไลบรารีเว็บแอดมิน 45
ก.2.4 ไลบรารีสร้างโมเดล 46
ก.2.5 ไลบรารี API Model 46
ภาคผนวก ข การทำงานส่วนของ Server 47
ข.1 การทำงานของ Server ส่วนการติดต่อกับ MQTT 47
ข.2 การทำงานของ Server ส่วนการติดต่อกับ API Model 51

สารบัญตาราง

หน้า

ตารางที่ 4.1 ขั้นตอนและแผนการดำเนินงาน 17
ตารางที่ 5.1 จำนวนชุดข้อมูลที่ได้ในแต่ละครั้ง 33

สารบัญรูป

หน้า

รูปที่ 2.1 ภาพรวมการเชื่อมต่อของระบบ (Wiring Diagram) 4
รูปที่ 2.2 ภาพระบบตรวจวัดอุณหภูมิร่างกาย 5
รูปที่ 2.3 ขั้นตอนการเก็บผลจากกล้องตรวจจับความร้อน AMG8833 6
รูปที่ 3.1 ESP8266 ESP-12E 8
รูปที่ 3.2 โมดูล ESP32-CAM 9
รูปที่ 3.3 เซนเซอร์ AMG8833 10
รูปที่ 3.4 ตัวอย่างการทำงานของ MQTT 11
รูปที่ 3.5 Concept หลักของ React 13
รูปที่ 3.6 ขั้นตอนการที่จะนำข้อมูลผู้ใช้มาใช้งานในเซิร์ฟเวอร์ของนักพัฒนา 15
รูปที่ 5.1 โครงสร้างการทำงานของระบบ 18
รูปที่ 5.2 แผนผังเว็บไซต์ของผู้ใช้ 20
รูปที่ 5.3 แผนผังเว็บไซต์ของผู้ดูแลระบบ 21
รูปที่ 5.4 แผนผังการทำงานของอุปกรณ์ AMG8833 และการแจ้งเตือนไลน์ 22
รูปที่ 5.5 แผนผังการทำงานของอุปกรณ์ ESP32-CAM 23
รูปที่ 5.6 แผนผังการทำงานการบันทึกภาพไปยังฐานข้อมูลของผู้ใช้ 23
รูปที่ 5.7 แผนผังการทำงานของโมเดล 24
รูปที่ 5.8 หน้าเริ่มต้นของเว็บไซต์ของผู้ใช้ 25
รูปที่ 5.9 หน้าล็อกอินไลน์ 22
รูปที่ 5.10 หน้าอนุญาติสิทธิ์การเข้าถึง 25
รูปที่ 5.11 หน้าหลักของเว็บไซต์ของผู้ใช้ 23
รูปที่ 5.12 หน้าเพิ่มอุปกรณ์ส่วนของผู้ใช้ 26
รูปที่ 5.13 หน้าหลักของเว็บไซต์ของผู้ใช้หลังมีการเพิ่มอุปกรณ์ 26
รูปที่ 5.14 หน้าใส่ Token 27
รูปที่ 5.15 หน้าแก้ไขชื่อบ้านของผู้ใช้ 27
รูปที่ 5.16 หน้าแก้ไขชื่ออุปกรณ์ของผู้ใช้ 28
รูปที่ 5.17 หน้าลบอุปกรณ์ของผู้ใช้ 28
รูปที่ 5.18 หน้าเลือกวันที่ดูประวัติของผู้ใช้ 26
รูปที่ 5.19 หน้าดูรูปประวัติการเกิดเหตุ 29
รูปที่ 5.20 หน้ารายงานปัญหาของผู้ใช้ 29
รูปที่ 5.21 หน้าแรกเว็บไซต์ของผู้ดูแลระบบ 30
รูปที่ 5.22 หน้าสมัครสมาชิกของผู้ดูแลระบบ 30
รูปที่ 5.23 หน้าล็อกอินของผู้ดูแลระบบ 30
รูปที่ 5.24 หน้าหลักเว็บไซต์ของผู้ดูแลระบบ 31
รูปที่ 5.25 หน้าเพิ่มอุปกรณ์ของผู้ดูแลระบบ 31

รูปที่ 5.26 หน้ารายงานปัญหาของผู้ดูแลระบบ 31
รูปที่ 5.27 ขั้นตอนการทดลองเก็บข้อมูล 32
รูปที่ 5.28 ค่าอุณหภูมิที่ได้จากการทดลองเก็บข้อมูล 34
รูปที่ 5.29 สถานการณ์ปกติ 34
รูปที่ 5.30 สถานการณ์ที่เริ่มมีความร้อน 34
รูปที่ 5.31 สถานการณ์ที่มีความร้อนสูงมากเกิน 34
รูปที่ 5.32 import library 35
รูปที่ 5.33 read csv 35
รูปที่ 5.34 read csv 35
รูปที่ 5.35 การแบ่งข้อมูล 35
รูปที่ 5.36 การกำหนดพารามิเตอร์ 35
รูปที่ 5.37 การกำหนดพารามิเตอร์ 35
รูปที่ 5.38 การกำหนดอินเทอร์เน็ตที่เชื่อม กำหนด URL และกำหนดคีย์ของ Firebase 36
รูปที่ 5.39 การกำหนด path และ กำหนดเวลาในการจับภาพ 36
รูปที่ 5.40 ภาพรูปแบบ base64 36
รูปที่ 5.41 ภาพที่ได้จากนำ base64 มาแสดง 37
รูปที่ 6.1 ผลการทดลองโมเดลจำนวนต้นไม้ 50 ต้น 40
รูปที่ 6.2 ผลการทดลองโมเดลจำนวนต้นไม้ 100 ต้น 40
รูปที่ 6.3 ผลการทดลองโมเดลจำนวนต้นไม้ 150 ต้น 40
รูปที่ 6.4 ผลการทดลองโมเดลจำนวนต้นไม้ 200 ต้น 40
รูปผนวกที่ ก.1.1 การเพิ่มโมดูลในโปรแกรม Arduino IDE 43
รูปผนวกที่ ก.1.2 ไลบรารีโมดูล ESP8266 กับ AMG8833 43
รูปผนวกที่ ก.1.3 ไลบรารีโมดูล ESP32-CAM 44
รูปผนวกที่ ก.2.1 ไลบรารีเซิร์ฟเวอร์ 44
รูปผนวกที่ ก.2.2 ไลบรารีเว็บผู้ใช้ 45
รูปผนวกที่ ก.2.3 ไลบรารีเว็บแอดมิน 45
รูปผนวกที่ ก.2.4 ไลบรารีสร้างโมเดล 46
รูปผนวกที่ ก.2.5 ไลบรารี API Model 46
รูปผนวกที่ ข.1.1 การเชื่อมต่อกับ MQTT Broker 47
รูปผนวกที่ ข.1.2 สมัครรับข้อมูล 48
รูปผนวกที่ ข.1.3 รับข้อมูลจากหัวข้อที่สมัครรับ 49
รูปผนวกที่ ข.1.4 ส่งข้อมูลไปยัง API model 49
รูปผนวกที่ ข.1.5 ค้นหา Token LINE Notify 50
รูปผนวกที่ ข.1.6 ส่งแจ้งเตือนไปยัง LINE 50
รูปผนวกที่ ข.1.7 บันทึกข้อมูลไปยัง Firestore Database 51
รูปผนวกที่ ข.2.1 API Model 51

บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญของปัญหา

ในปัจจุบันเครือข่ายอินเทอร์เน็ตได้เข้ามามีส่วนร่วมในการใช้ชีวิตประจำวัน กล่าวได้ว่าประชากรส่วนใหญ่สามารถเข้าถึงอินเทอร์เน็ตได้อย่างทั่วถึง อีกทั้งยังมีเทคโนโลยี Internet of Things (IoT) หรือ “อินเทอร์เน็ตของสรรพสิ่ง” เป็นสิ่งที่ควบคู่กันมากับเครือข่ายอินเทอร์เน็ต ซึ่งเทคโนโลยีนี้ สามารถสั่งการหรือเก็บข้อมูลระยะไกล เช่น การสั่งการเครื่องใช้ในบ้าน การเก็บข้อมูลพนักงานภายในบริษัท หรือการสั่งการเครื่องใช้ในชีวิตประจำวันต่าง ๆผ่านทางอินเทอร์เน็ตโดยใช้ตัวเซนเซอร์หรือตัวเก็บข้อมูลซึ่งฝังอยู่ในวัตถุต่าง ๆ โดยสิ่งเหล่านี้จะสามารถสร้างความได้เปรียบในการรับข้อมูลหรือออกได้ Realtime และแม่นยำมากขึ้น

ปัญหาการเกิดเพลิงไหม้ภายในที่พักอาศัย คือ ภัยใกล้ตัวที่เกิดขึ้นโดยสาเหตุส่วนใหญ่มาจากความประมาท ความรู้เท่าไม่ถึงการณ์ รวมถึงการหลงลืม เช่น การลืมปิดเครื่องใช้ไฟฟ้าขณะที่ไม่อยู่ภายในที่พักอาศัยเป็นสาเหตุทำให้เกิดไฟฟ้าลัดวงจร การใช้สายไฟผิดประเภท และการใช้เครื่องใช้ไฟฟ้าที่กินกำลังเกินที่สายไฟจะรับได้ ซึ่งการเกิดเพลิงไหม้แต่ละครั้งก่อให้เกิดความสูญเสียต่อชีวิตและก่อให้เกิดความเสียหายแก่ทรัพย์สิน

จากปัญหาที่กล่าวมาข้างต้นผู้วิจัยมีแนวคิดในการพัฒนาระบบแจ้งเตือนการเกิดเพลิงไหม้ในจุดที่เสี่ยงต่อการเกิดเพลิงไหม้ภายในที่พักอาศัย โดยมีการประยุกต์ใช้อุปกรณ์อินเทอร์เน็ตของสรรพสิ่ง Internet of Things (IoT) ซึ่งอุปกรณ์มีราคาถูก มีประสิทธิภาพและหาได้ง่าย ใช้สำหรับตรวจจับอุณหภูมิของสายไฟในจุดที่เสี่ยงต่อการเกิดเพลิงไหม้ และแจ้งเตือนผ่านแอปพลิเคชันไลน์(Application LINE) แบบอัตโนมัติจะทำให้แก้ไขปัญหาดังกล่าวได้ทันก่อนที่จะเกิดเหตุเพลิงไหม้รุนแรง โดยการตรวจจับอุณหภูมิจะใช้เซนเซอร์กล้องอินฟราเรด AMG8833 ร่วมกับบอร์ด ESP8266 และ โมดูลกล้อง ESP32-CAM สำหรับคำสั่งการมีระบบวิเคราะห์อุณหภูมิเพื่อให้แน่ใจว่ามีอุณหภูมิสูงขึ้นผิดปกติตรงจุดที่ติดตั้งอุปกรณ์ไว้หรือไม่ หากระบบวิเคราะห์อุณหภูมิแล้วพบว่ามีอุณหภูมิสูงเกินกว่าปกติ ระบบจะทำการแจ้งเตือนผ่านแอปพลิเคชันไลน์แบบอัตโนมัติแจ้งเตือนไปยังผู้ใช้ ทำให้ผู้ใช้สามารถแก้ไขปัญหาได้ทันก่อนเกิดเหตุเพลิงไหม้

1.2 วัตถุประสงค์

1) เพื่อศึกษาและพัฒนาระบบแจ้งเตือนการเกิดเพลิงไหม้ในจุดที่เสี่ยงต่อการเกิดเพลิงไหม้ภายในที่พักอาศัย โดยใช้หลักการอินเทอร์เน็ตของสรรพสิ่ง Internet of Things (IoT) เพื่อตรวจสอบความผิดปกติที่เกิดขึ้น
2) เพื่อประเมินความพึงพอใจของผู้ใช้

1.3 ลักษณะและขอบเขต

โครงงานวิจัยนี้เป็นการพัฒนาระบบอินเทอร์เน็ตของสรรพสิ่ง Internet of Things (IoT) สำหรับแจ้งเตือนการเกิดเพลิงไหม้ในจุดที่เสี่ยงต่อการเกิดเพลิงไหม้ภายในที่พักอาศัย ดังนั้นปัจจัยที่มีผลต่อการตรวจจับอุณหภูมิในจุดที่เสี่ยง มีปัจจัยสำคัญต่อไปนี้

ผู้ใช้

- ทำการติดตั้งอุปกรณ์ เซนเซอร์กล้องอินฟราเรด AMG8833 และโมดูลกล้อง ESP32-CAM
- ทำการลงทะเบียนโดยเข้าสู่ระบบไลน์

- ทำการเพิ่มชื่อของจุดที่ทำการติดตั้ง และชื่ออุปกรณ์
- สามารถเพิ่ม ลบ และแก้ไขอุปกรณ์ได้
- ตรวจสอบตรงจุดที่ทำการติดตั้งอุปกรณ์หลังจากที่มีการส่งแจ้งเตือนไปยังแอปพลิเคชันไลน์ (Application LINE) ว่ามีเหตุการณ์ผิดปกติจริงหรือไม่ และทำการแก้ปัญหาหากมีความผิดปกติจริง

ผู้ดูแลระบบ

- ตั้งค่าตัวอุปกรณ์หลังจากที่ผู้ใช้ติดตั้งอุปกรณ์เสร็จ
- ออกแบบระบบการลงทะเบียน โดยการระบุชื่อจุดที่ติดตั้ง และหมายเลขเครื่องที่นำไปติดตั้ง
- ออกแบบระบบวิเคราะห์ข้อมูลที่ได้รับจากอุปกรณ์
- ออกแบบระบบแจ้งเตือนผู้ใช้ โดยจะแจ้งเตือนผ่านแอปพลิเคชันไลน์

1.4 อุปกรณ์และเครื่องมือที่ใช้

1.4.1 ฮาร์ดแวร์

- Client
  - บอร์ด ESP8266
  - เซนเซอร์กล้องอินฟราเรด AMG8833
  - โมดูลกล้อง ESP32-CAM
- Server
  - CPU: Intel(R) Core (TM) i5-9300H CPU @ 2.40GHz 2.40 GHz
  - Memory: 20 GB
  - Hard disk: 220 GB

1.4.2 ซอฟต์แวร์

- Client
  - ระบบปฏิบัติการวินโดว์ 11
- Server
  - Arduino IDE
  - Visual Studio Code
  - Back-End: Node JS, Express JS
  - Front-End: React
  - Database : Firebase

1.5 ผลที่คาดว่าจะได้รับ

1) ระบบสามารถส่งแจ้งเตือนก่อนเกิดเหตุเพลิงไหม้ โดยใช้หลักการอินเทอร์เน็ตของสรรพสิ่ง Internet of Things (IoT)
2) อุปกรณ์สามารถตรวจจับอุณหภูมิตรงจุดที่ติดตั้งอุปกรณ์ได้อย่างถูกต้องและรวดเร็ว
3) ระบบสามารถส่งแจ้งเตือนไปยังผู้ใช้ได้อย่างรวดเร็ว ถูกต้อง และทันต่อการแก้ไขปัญหา

1.6 ภาระหน้าที่ความรับผิดชอบ

1) นางสาว สุพิชชา เมธากิจภาคิน

   <u>รับผิดชอบ</u> ต่ออุปกรณ์ เขียนโปรแกรม Arduino IDE, MQTT

2) นาย ปรัชญา สุวรรณสุข

   <u>รับผิดชอบ</u> ฐานข้อมูล เว็ปแอพพลิเคชัน Server และทำโมเดล Machine Learning

3) ความรับผิดชอบร่วม

   <u>รับผิดชอบ</u> เก็บรวบรวมข้อมูลและทำการทดลอง

บทที่ 2 ผลงานที่เกี่ยวข้อง

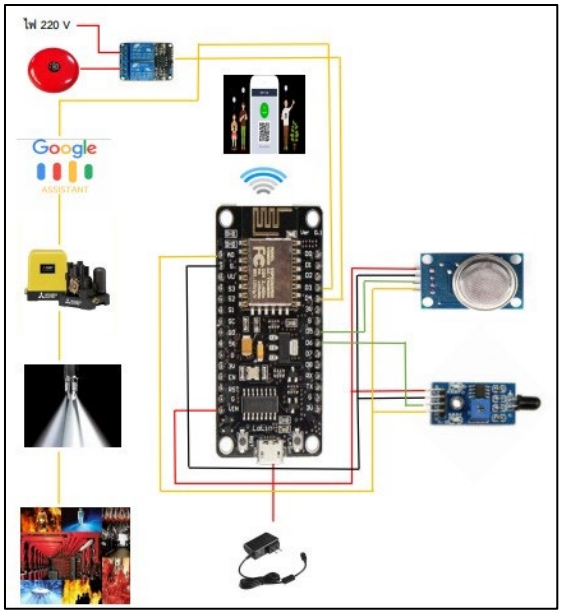
เป็นบททที่กล่าวถึงผลงานต่าง ๆที่เกี่ยวข้องกับโครงงานปริญญานิพนธ์นี้ ซึ่งจะอธิบายถึงคุณลักษณะข้อดี และข้อเสียของแต่ละผลงาน โดยมีผลงานที่เกี่ยวข้องกับโครงงานปริญญานิพนธ์มีดังนี้

2.1 ชื่อวิจัย : เครื่องแจ้งเตือนและควบคุมอัคคีภัยผ่าน LINE Notify [1]

โดย : นรจ.ชาญณรงค์ สมเพ็ชร, นรจ.พชร ทองฉิม และคณะผู้จัดทำ

โครงงานนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรนักเรียนจ่าทหารเรือพรรคพิเศษ เหล่าช่างยุทธโยธาอิเล็กทรอนิกส์ ปีการศึกษา 2561

คณะผู้จัดทำเล็งเห็นปัญหาที่เกิดขึ้นภายในอาคารสถานที่มีหลายรูปแบบ เช่นการเกิดอัคคีภัย ซึ่งเกิดจากหลายสาเหตุด้วยกัน และที่ผ่านมาก็พบว่าสาเหตุส่วนใหญ่เกิดจากความประมาทและการมองข้ามสิ่งเล็กน้อย ดังนั้นทางผู้จัดทำโครงงานจึงเห็นว่าควรจะนำเทคโนโลยี สมัยใหม่มาประยุกต์ใช้กับระบบความปลอดภัยภายในบ้าน ทางผู้จัดทำโครงงานมีความประสงค์ที่จะสร้างเครื่องมือเพื่อช่วยเพิ่มความปลอดภัยภายในบ้าน โดยการตรวจจับสาเหตุของภัยที่จะเกิดขึ้นภายในบ้าน ด้วยวงจรอิเล็กทรอนิกส์และไมโครคอนโทรเลอร์แจ้งเหตุเมื่อเกิดภัย จึงได้นำอุปกรณ์ที่สามารถตรวจจับกลุ่มของควันไฟที่ก่อตัวขึ้น การรั่วไหลของแก๊สหุงต้มภายในอาคารสถานที่โดยใช้อุปกรณ์ตรวจจับควัน อุปกรณ์ตรวจจับความร้อน ซึ่งอุปกรณ์ต่อไปนี้สามารถแจ้งเตือน ไปยังแอพพลิเคชั่นภายในโทรศัพท์เคลื่อนที่ของผู้ใช้ได้



รูปที่ 2.1 ภาพรวมการเชื่อมต่อของระบบ (Wiring Diagram)

จากรูปที่ 2.1 การทำงานของอุปกรณ์เมื่อเกิดไฟไหม้ Sensor 2 ตัวได้แก่ Smoke Detector และ Heat Detector ซึ่งเป็นตัวตรวจจับควัน และ ความร้อนจะทำการตรวจจับค่าควันและความร้อนที่เกินปริมาณ ซึ่งอาจจะทำให้เกิดเพลิงไหม้ได้ จากนั้นจะส่งค่า ข้อมูลเข้าไปยัง Arduino ESP 8266 Node MCU และบอร์ด Arduino จะทำการประมวลผลหากความหนาแน่น ของควันไฟและความร้อนเกินกว่าค่าที่ได้กำหนดไว้จึงจะส่งสัญญาณและทำการส่ง output ออกไปยัง Relay เพื่อกระตุ้นให้ Relay จ่ายไฟ 220V ให้กับกระดิ่งเตือนภัยทำงาน ในขณะเดียวกันยังสามารถส่งข้อความแจ้งเตือน ไปยัง แอปพลิเคชันไลน์ บนอุปกรณ์สื่อสารได้ และสามารถติดตั้งแอปพลิเคชันส่งสัญญาณไปควบคุม Sprinkler หรือ CO2 Valve เพื่อควบคุมเพลิงได้

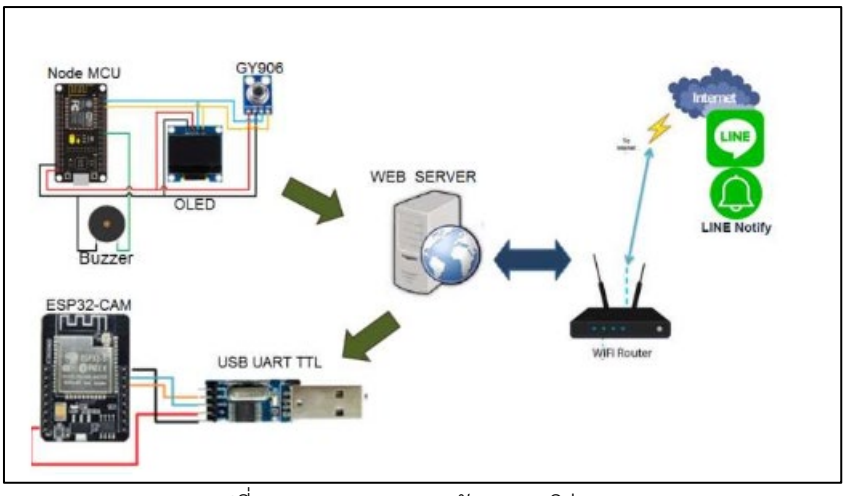
ข้อดีของระบบ สามารถตรวจจับควันและความร้อนได้หากตรวจจับได้ว่าค่าควันและความร้อนเกินปริมาณจะทำการแจ้งเตือนให้เจ้าของบ้านให้รับรู้ถึงสถานการณ์ล่วงหน้าก่อนมีเหตุเพลิงไหม้ โดยจะแจ้งเตือนผ่านแอปพลิเคชันไลน์ และสามารถสั่งให้ Sprinkler ทำงานได้โดยสั่งจากแอปพลิเคชัน

2.2 งานวิจัย : การพัฒนาระบบตรวจวัดอุณหภูมิร่างกายแจ้งเตือนและภาพถ่ายแอพพลิเคชั่นไลน์ [2]

โดย : ธีรถวัลย์ ปานกลาง, รักเกียรติ ขำดำรงเกียรติ และรัตนสุดา สุภดนัยสร

การพัฒนาระบบตรวจวัดอุณหภูมิร่างกาย แจ้งเตือนข้อความและภาพถ่ายผ่านแอพพลิเคชั่นไลน์

โดยคณะผู้จัดได้พัฒนาระบบตรวจวัดอุณหภูมิร่างกายแจ้งเตือนข้อความและภาพถ่ายผ่านแอพพลิเคชั่นไลน์เพื่ออำนวยความสะดวกช่วยให้แพทย์ สามารถตรวจวัดอุณหภูมิของผู้ป่วยได้โดยไม่ต้องสัมผัส และยังสามารถส่งข้อความและภาพถ่ายแจ้งเตือนเมื่อพบความผิดปกติของอุณหภูมิร่างกาย ผ่านแอพพลิเคชั่นไลน์ สามารถจัดเก็บข้อมูลรวมถึงการสืบค้นข้อมูลการตรวจวัดอุณหภูมิร่างกายย้อนหลังตามวันที่ผู้ใช้งานต้องทราบได้ ผ่านเครือค่ายอินเทอร์เน็ต เพื่ออำนวยความสะดวกในการวินิจฉัยโรคและติดตามผู้ป่วยได้รวดเร็ว



รูปที่ 2.2 ภาพระบบตรวจวัดอุณหภูมิร่างกาย

จากรูปที่ 2.2 ระบบตรวจวัดอุณหภูมิร่างกายกายนี้จะใช้ตัวอินฟราเรดเซนเซอร์สำหรับตรวจจับความร้อน และส่งค่าเอาต์พุตที่ได้จากเซนเซอร์ไปยังไมโครคอนโทรลเลอร์ Node MCU V.2 เพื่อทำการประมวลผลค่าอุณหภูมิส่งค่าที่ได้ไปแสดงผลบนจอ OLED พร้อมกับจัดเก็บผลอุณหภูมิลงในฐานข้อมูล สามารถเลือกดูข้อมูลอุณหภูมิย้อนหลัง ร่วมกับโมดูลกล้อง ESP32-CAM ซึ่งทำหน้าที่ถ่ายภาพและส่งภาพพร้อมข้อความแจ้งเตือนไปยังแอพพลิเคชั่นไลน์เมื่อพบความผิดปกติของอุณหภูมิ

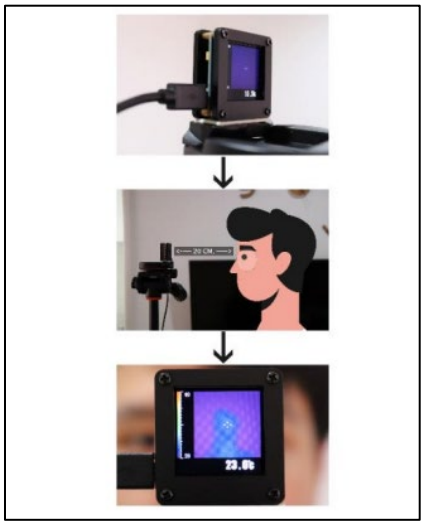
ร่างกาย ซึ่งระยะที่เหมาะสมคือระยะห่าง 3 cm โดยมีค่าความผิดพลาด 0.07 องศาเซลเซียส มีการวิเคราะห์แปลผลอุณหภูมิสถานะปกติ – สถานะมีไข้จัดเก็บในฐานข้อมูลบน Web Server ซึ่งสามารถตรวจสอบค่าอุณหภูมิย้อนหลังผ่านหน้าเว็บเพจ

2.3 งานวิจัย : ระบบคัดกรองผู้ที่มีอาการป่วยในที่สาธารณะ [3]

โดย : กมลพัชร ใจเพชร และ พงศ์ปณต ตั้งตราช

สาขาวิชาเทคโนโลยีมีเดีย คณะสถาปัตยกรรมศาสตร์และการออกแบบ มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

งานวิจัยนี้จะนำเสนอระบบคัดกรองผู้ที่มีอาการป่วยในที่สาธารณะและแจ้งเตือนผู้ที่อยู่โดยรอบด้วยระบบเครือข่ายมือถือ(Patient in Public Screening System and Notification to Nearby People with Mobile Network)ซึ่งเป็นระบบคัดกรองผู้ที่มีอาการป่วยเบื้องต้น ด้วยการนำเทคโนโลยีการรู้จำใบหน้ารวมเข้ากับเทคโนโลยีกล้องตรวจจับความร้อนเพื่อช่วยลดความเสี่ยง และยับยั้งการแพร่ระบาดโรค COVID-19 โดยแนวคิดคือ การใช้กล้องตรวจจับความร้อนตรวจจับไปยังบุคคลที่เดินผ่านกล้อง พร้อมกับบันทึกอุณหภูมิที่ได้ในฐานข้อมูลของแต่ละบุคคล จากการตรวจจับใบหน้า หากพบผู้ที่มีอุณหภูมิร่างกายมากกว่า 37.5 องศาเซลเซียส ระบบจะแจ้งเตือนบุคคลที่อยู่โดยรอบผู้ที่มีอาการป่วยในรัศมี 5-10 เมตร ผ่านระบบเครือข่ายมือถือ เพื่อให้บุคคลที่อยู่โดยรอบป้องกันตัว ในการตรวจสอบประสิทธิภาพของระบบนั้นได้ทำการใช้กล้องเซ็นเซอร์ตรวจจับความร้อน AMG8833 ตรวจจับอุณหภูมิของบุคคลเปรียบเทียบความคลาดเคลื่อนอุณหภูมิกับการใช้เครื่องวัดอุณหภูมิร่างกายแบบดิจิทัลที่ใช้ในทางการแพทย์ โดยกล้องเซ็นเซอร์ตรวจจับความร้อนมีความคลาดเคลื่อน 24.07 เปอร์เซ็นต์ เมื่อเปรียบเทียบกับเครื่องวัดอุณหภูมิร่างกายแบบดิจิทัลที่ใช้ในทางการแพทย์



รูปที่ 2.3 ขั้นตอนการเก็บผลจากกล้องตรวจจับความร้อน AMG8833

วิธีการวิจัยเริ่มต้นโดยการใช้กล้องตรวจจับความร้อน AMG8833 ติดบนขาตั้งเซ็นเซอร์ตรวจจับความร้อนตรงกับบริเวณหน้าผากของมนุษย์จากนั้นให้ผู้ทดลองเดินเข้ามาบริเวณกล้องโดยมีความห่างจากกล้องตรวจจับความร้อนโดยประมาณ 20 เซนติเมตร ผู้วิจัยจึงอ่านค่าที่ได้และบันทึกผลการทดลอง กล้องตรวจจับความร้อนจะทำการแยกแยะอุณหภูมิที่วัดได้ในตำแหน่งนั้น ๆ หากอุณหภูมิต่ำพิกเซลบริเวณนั้นจะแสดงผลเป็นสีฟ้า-ม่วง หากอุณหภูมิบริเวณนั้น ๆ สูงพิกเซลจะแสดงผลสีเหลือง - แดง ทั้งนี้จะขึ้นอยู่กับการตั้งค่าอุณหภูมิสูงสุดและต่ำสุดของกล้องตรวจจับความร้อนหลังจากวัดผลรากฏว่า ผู้เข้าร่วมการทดลองจำนวน 5 คน มีผลการวัดอุณหภูมิร่างกายโดยใช้เครื่องวัดแบบดิจิทัล ค่าเฉลี่ยอยู่ที่ 35.98 องศาเซลเซียส และมีผลวัดอุณหภูมิร่างกายโดยใช้กล้องตรวจจับความร้อน AMG8833 ค่าเฉลี่ยอยู่ที่ 27.32 องศาเซลเซียส โดยมีค่าผิดพลาดเฉลี่ย ร้อย

ละ 24.07 ตามลำดับ ซึ่งความผิดพลาดที่เกิดขึ้นอาจมีสาเหตุมาจากวิธีการทำงานของอุปกรณ์ที่แตกต่างกันออกไป โดยกล้องตรวจจับความร้อน AMG8833 เป็นการใช้เซ็นเซอร์วัดอุณภูมิพื้นผิวของวัสดุนั้น ซึ่งหากนำมาวัดอุณหภูมิร่างกายนั้น อาจเป็นการวัดผลจากอุณหภูมิของผิวหนัง

บทที่ 3 ทฤษฎีและความรู้ที่เกี่ยวข้อง

การจัดทำโครงงานวิจัยเรื่องการพัฒนาระบบแจ้งเตือนการเกิดเพลิงไหม้ด้วยอินเทอร์เน็ตของสรรพสิ่ง คณะผู้จัดทำได้ค้นคว้าข้อมูลทฤษฎีและความรู้เกี่ยวกับโครงงาน ซึ่งมีเนื้อหาดังนี้

3.1 NodeMCU ESP8266 V3 [4]

ESP8266 คือ โมดูล Wi-Fi ที่สามารถโปรแกรมลงไปได้ ทำงานที่แรงดันไฟฟ้า 3.3V - 3.6V การนำไปใช้งานร่วมกับเซนเซอร์อื่น ๆ ที่ใช้แรงดัน 5V ต้องใช้วงจรแบ่งแรงดันมาช่วย เพื่อไม่ให้โมดูลพัง ขาของโมดูล ESP8266 แบ่งได้ดังนี้

1. VCC เป็นขาสำหรับจ่ายไปเข้าเพื่อให้โมดูลทำงานได้ ซึ่งแรงดันที่ใช้งานได้คือ 3.3 - 3.6V

2. GND

3. Reset และ CH_PD (หรือ EN) เป็นขาที่ต้องต่อเข้าไฟ + เพื่อให้โมดูลสามารถทำงานได้ ทั้ง 2 ขานี้ สามารถนำมาใช้รีเซ็ตโมดูลได้เหมือนกัน แตกต่างตรงที่ขา Reset สามารถลอยไว้ได้ แต่ขา CH_PD (หรือ EN) จำเป็นต้องต่อเข้าไป + เท่านั้น เมื่อขานี้ไม่ต่อเข้าไฟ + โมดูลจะไม่ทำงานทันที

4. GPIO เป็นขาดิจิตอลอินพุต / เอาต์พุตทำงานที่แรงดัน 3.3V

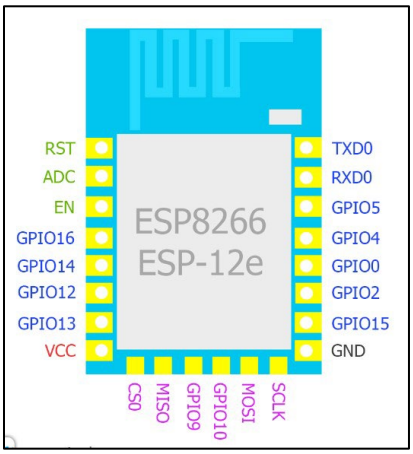
5. GPIO15 เป็นขาที่ต้องต่อลง GND เท่านั้น เพื่อให้โมดูลทำงานได้

6. GPIO0 เป็นขาทำหรับการเลือกโหมดทำงาน หากนำขานี้ลง GND จะเข้าโหมดโปรแกรม หากลอยไว้ หรือนำเข้าไฟ + จะเข้าโหมดการทำงานปกติ

7. ADC เป็นขาอนาล็อกอินพุต รับแรงดันได้สูงสุดที่ 1V ขนาด 10 บิต

ตัวโมดูลสามารถเขียนโปรแกรมเพื่อควบคุมและสั่งการการทำงาน และยังสามารถเขียนโปรแกรมควบคุมใช้ งานขา GPIO บนโมดูลได้ ทำให้สามารถนำ sensor ต่าง ๆ มาต่อพ่วงเข้าไป เช่น อุณหภูมิความชื้น แสงสว่าง ค่า ความดัน เป็นต้น โดยใช้ภาษา C,C++ และเขียนโปรแกรมได้จาก Arduino IDE ได้

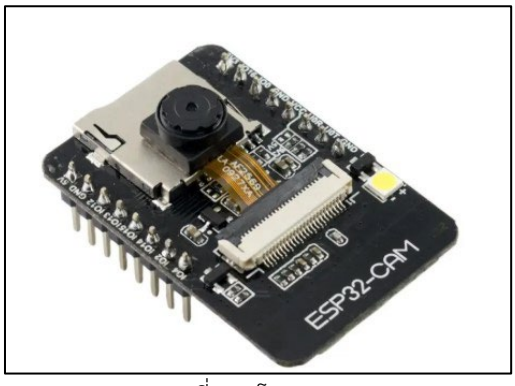
โครงงานนี้จะใช้โมดูล ESP8266 มาใช้คู่กับเซนเซอร์ AMG8833 ซึ่งใช้ในการรับข้อมูลอุณหภูมิจากเซนเซอร์ AMG8833 และส่งข้อมูลอุณหภูมิที่ได้ไปยัง HiveMQ



รูปที่ 3.1 ESP8266 ESP-12E

ที่มา (https://www.artronshop.co.th)

3.2 ESP32 – CAM [5]



รูปที่ 3.2 โมดูล ESP32-CAM

ที่มา (https://techtalk2apply.com /)

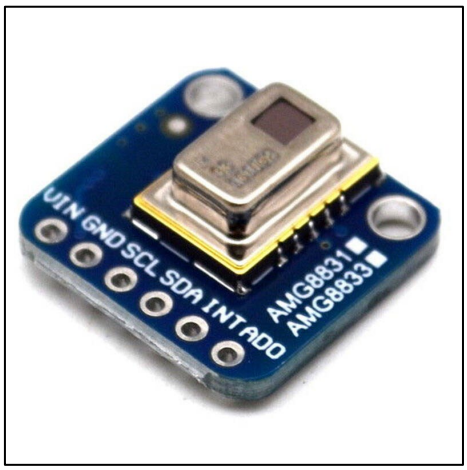
ESP32-CAM เป็นโมดูลที่คุณสามารถใช้กับโครงการต่าง ๆมากมายและกับ Arduino เป็นโมดูลที่สมบูรณ์พร้อมด้วยไมโครคอนโทรลเลอร์ในตัวซึ่งสามารถทำให้ทำงานได้อย่างอิสระ โมดูลนี้มีกล้องวิดีโอในตัวและช่องเสียบ microSD สำหรับจัดเก็บข้อมูล

คุณสมบัติของ ESP32-CAM

- โมดูลออนบอร์ด ESP32-S รองรับ Wi-Fi + Bluetooth
- กล้อง OV2640 พร้อมแฟลชในตัว
- แฟลชในตัว: 32Mbit
- RAM: ภายใน 512KB + ภายนอก 4M PSRAM
- เสาอากาศ: เสาอากาศ PCB ออนบอร์ด
- โปรโตคอล Wi-Fi: IEEE 802.11 b/g/n/e/i
- บลูทูธ : บลูทูธ 4.2 BR/EDR และ BLE
- โหมด WIFI: Station / SoftAP / SoftAP + Station
- ความปลอดภัย: WPA/WPA2/WPA2-Enterprise/WPS
- รูปแบบภาพเอาต์พุต: JPEG (รองรับ OV2640 เท่านั้น), BMP, GREYSCALE
- การ์ด TF ที่รองรับ: สูงสุด 4G
- อินเทอร์เฟซอุปกรณ์ต่อพ่วง: UART/SPI/I2C/PWM
- พอร์ต IO: 9
- อัตรารับส่งข้อมูล UART: ค่าเริ่มต้น 115200bps
- แหล่งจ่ายไฟ: 5V pinout ของ ESP32-CAM กล้องจะเชื่อมต่อกับขั้วต่อที่เปิดใช้งาน

โครงงานนี้ใช้โมดูล ESP32-CAM ในการจับภาพเพื่อเป็นหลักฐานยืนยันแก่ผู้ใช้

3.3 เซนเซอร์ AMG8833 [6]



รูปที่ 3.3 เซนเซอร์ AMG8833

(ที่มา Thermal Camera Analysis with Raspberry Pi (AMG8833) — Maker Portal (makersportal.com)

AMG8833 เป็นอาร์เรย์ 8x8 ของเซนเซอร์ความร้อน IR เมื่อเชื่อมต่อกับไมโครคอนโทรลเลอร์ มันจะส่งคืนค่าอาร์ของการอ่านอุณหภูมิอินฟราเรด 64 พิกเซล ผ่าน I2C วัดอุณหภูมิได้ตั้งแต่ได้ตั้งแต่ 0°C ถึง 80 °C ความแม่นยำ + - 2.5 °C สามารถตรวจจับได้ไกลจากตัวเซนเซอร์ 7 เมตร ด้วยอัตราเฟรมสูงสุด 10 Hz จึงเหมาะอย่างยิ่งในการนำมาสร้างเครื่องตัวจัดมนุษย์หรือกล้องจับความร้อนขนาดเล็กคุณสมบัติของ AMG8833

- การตรวจจับอุณหภูมิของพื้นที่สองมิติ 8x8 (64 พิกเซล)
- เอาต์พุต I2C (ความสามารถของเอาต์พุตค่าอุณหภูมิ)
- รองรับทั้ง Arduino และราสเบอร์รี่ Pi
- ช่วงอุณหภูมิของวัตถุวัด: 0 ° C ถึง 80 ° C
- แพ็คเกจ SMD ขนาดกะทัดรัด (ปรับให้เข้ากับการติดตั้งแบบ reflow)เป็นไปตามข้อกำหนด RoHS

โครงงานนี้ใช้เซนเซอร์ AMG8833 ต่อเข้ากับโมดูล ESP8266 ซึ่งเซนเซอร์ AMG8833 จะทำหน้าที่ตรวจจับอุณหภูมิ

3.4 Arduino IDE [7]

ซอฟต์แวร์ที่ใช้ในการพัฒนางานสำหรับบอร์ด Arduino นั่นคือโปรแกรมที่เรียกว่า Arduino IDE ใช้ในการเขียนโปรแกรมและคอมไพล์ลงบอร์ด Arduino หรือบอร์ดตัวอื่น ๆที่คล้ายกัน เช่น Generic ESP8266 modules, NodeMCU หรือ WeMos D1 เป็นต้น

แนวคิดการใช้งานโปรแกรม Arduino IDE

1. เขียนโปรแกรมด้วยภาษา C/C++ สำหรับ Arduino IDE
2. คอมไพล์หรือแปลโปรแกรมภาษา C/C++ ให้เป็นภาษาสำหรับไมโครคอนโทรลเลอร์และบันทึกเป็น Intel Hex File
3. อัปโหลด Intel Hex File ลงบนไมโครคอนโทรลเลอร์ซึ่งอยู่บนบอร์ด Arduino ผ่านสาย USB หรือผ่าน Programmer

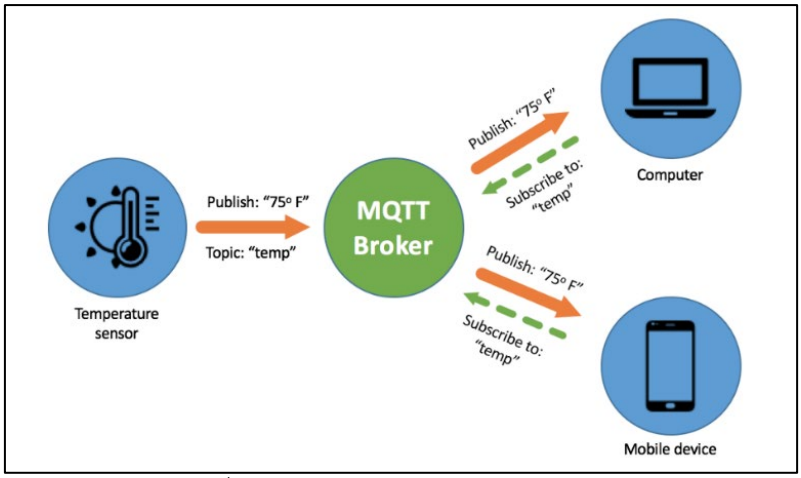
โครงงานนี้ใช้งานโปรแกรม Arduino IDE ในการเขียนโปรแกรมการทำงานของโมดูล ESP8266 และโมดูล ESP32-CAM

3.5 MQTT [8]

MQTT (Message Queue Telemetry Transport) เป็น Protocol ในการส่งข่าวสารหรือข้อความเล็ก ๆ โดยใช้โมเดลเน็ตเวิร์คแบบ publish-subscribe ที่ออกแบบมาสำหรับการสื่อสารระหว่าง machine to machine หรือ M2M มักจะส่งและรับข้อมูลผ่านเครือข่ายที่มีข้อจำกัดด้านทรัพยากร ซึ่งมี Bandwidth น้อย ถูกพัฒนาโดย Andy Stanford-Clark(IBM) และ Arlen Nipper ในปี 1999 ซึ่งเป็น โปรโตคอลที่ใช้เชื่อต่ออุปกรณ์ IoT ที่นิยมใช้มากที่สุดในปัจจุบัน เนื่องจากใช้งานง่าย การส่งข้อมูลจะไม่มีการ loss ระหว่างทางเพราะ โปรโตคอลจะรันอยู่บทเทคโนโลยี TCP/IP

หลักการทำงาน MQTT จะแบ่งบทบาทของอุปกรณ์ออกเป็น MQTT Broker และ MQTT Client

- MQTT Broker เป็นเซิฟเวอร์ซึ่งทำหน้าที่รับ message ที่ส่งมาจาก Client ตัวหนึ่งและส่งไปยัง Client อีกตัวหนึ่ง
- MQTT Client เป็นอุปกรณ์ที่ทำหน้าที่ส่งข้อความไปหา Broker และสามารถรับข้อความจาก Broker โดย Client นั่นจะเป็นอุปกรณ์ประเภทไหนก็ได้ เช่น Micro controller, โทรศัพท์มือถือ, คอมพิวเตอร์
- Publish / Subscribe การส่งข้อมูลเราจะต้องบอกด้วยว่าข้อมูลที่เราจะส่งไปนั้น เราจะส่งไปยัง Topic ไหน และการรับข้อมูลเราจะรับข้อมูลได้เฉพาะ Topic ที่เราได้ Subscribe เท่านั้น



รูปที่ 3.4 ตัวอย่างการทำงานของ MQTT
ที่มา (https://shorturl.asia/lwh6T)

- ตัวอย่างเช่น Temperature sensor เป็น MQTT Client เราจะเรียกว่าเป็น Publisher ซึ่งจะมีการประกาศหัวข้อ(Topic) ว่าจะส่งข้อมูลที่ชื่อ temp และจะมีการแนบข้อมูลมาคือ 75 ฟาเรนไฮต์ ส่งไปยัง MQTT Broker เพื่อส่งต่อข้อมูลไปยังอุปกรณ์ที่ทำการ Subscribe ในหัวข้อ temp ไว้ไม่ว่าจะมีกี่อุปกรณ์

โครงงานนี้มีการใช้ Protocol MQTT ในการส่งและรับข้อมูลจาก MQTT Broker โดยใช้โมดูล ESP8266 ในการ Publish ข้อมูลไปยัง MQTT Broker และใช้เซิร์ฟเวอร์ในการ Subscribe เพื่อรอรับข้อมูลจาก MQTT Broker

3.6 Node JS [9]

Node JSเป็นสภาพแวดล้อมการทำงานของภาษา JavaScript นอกเว็บเบราว์เซอร์ที่ทำงานด้วย V8 engine ซึ่งจะหมายความว่าเราสามารถใช้ Node JS ในการพัฒนาแอปพลิเคชันแบบ Command line แอปพลิเคชัน Desktop หรือแม้แต่เว็บเซิร์ฟเวอร์ได้ สามารถจัดการกับคำขอหลายอันได้พร้อมกัน มีไลบรารีมาตรฐานที่สามารถในมาใช้งานได้

โดยในโครงงานวิจัยนี้ได้มีการใช้ไลบรารีใน Node JS ดังนี้

- Express ใช้ในการสร้างแอปพลิเคชันเว็บไซต์และเซิร์ฟเวอร์ มีความสามารถในการจัดการการเริ่มต้นแอปพลิเคชันและการเรียก API
- React ใช้สำหรับสร้างอินเตอร์เฟสผู้ใช้ (UI) ในเว็บแอพพลิเชชัน และการตอบสนองผู้ใช้
- Axios ใช้ในการทำ HTTP requests ในเว็บแอปพลิเคชัน ช่วยเชื่อมต่อกับเว็บเซิร์ฟเวอร์เพื่อรับหรือส่งข้อมูล
- Firebase ใช้ในการพัฒนาเว็บเซิร์ฟเวอร์ มีเครื่องมือสำหรับการจัดการฐานข้อมูล
- MQTT ใช้ในการ subscribe ไปยัง MQTT Broker เพื่อรับข้อมูล

3.7 แอปพลิเคชันไลน์โนติฟาย (LINE Notify) [10]

LINE Notify เป็นบริการของทาง LINE โดยสามารถส่งข้อความการแจ้งเตือนต่าง ๆ ไปยังแอคเคาท์ (Account) ผ่านการใช้ API ซึ่งเรียกผ่าน HTTP POST แบบง่าย ๆ ข้อจำกัดของ LINE Notify คือ สามารถที่จะส่งแจ้งเตือนได้เฉพาะผู้ที่ขอใช้หรือกลุ่มผู้ขอใช้เป็นสมาชิกเท่านั้น ไม่สามารถส่งข้อความเข้าห้องสนทนาของเพื่อนๆ ได้หากต้องการให้สามารถส่งข้อความใช้ LINE Bot API ในส่วนของการใช้งาน LINE Notify เพิ่ม LINE Notify เป็นเพื่อนก่อนจะใช้งาน API และส่งการแจ้งเตือน ต้องเพิ่ม LINE Notify เป็นเพื่อนก่อน

โครงงานนี้ได้มีการใช้ LINE Notify ในการส่งแจ้งเตือนความผิดปกติที่เกิดขึ้น โดยการเรียก API ของ LINE Notify “https://notify-api.line.me/api/notify” พร้อมกับระบุ Token ที่ผู้ใช้ทำการออกมา เพื่อแจ้งเตือนไปยังผู้ใช้

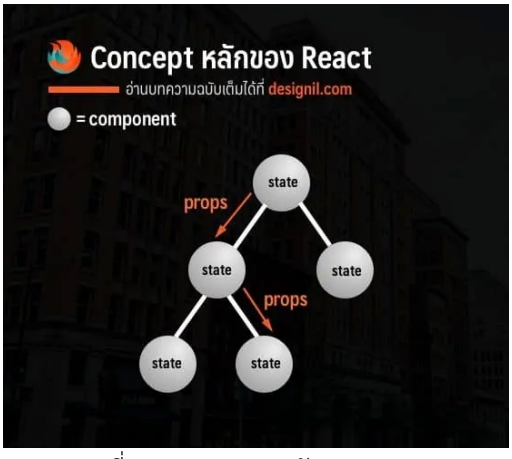
3.8 React [11]

React คือ JavaScript library ตัวนึงที่ถูกนำมาใช้สร้าง user interface พัฒนาขึ้นโดย Facebook ซึ่งจะช่วยให้เราเขียนโค้ดในการสร้าง UI ที่มีความซับซ้อนได้ React ยังมาช่วยลดขั้นตอนการทำงานของ Server ได้เป็นอย่างดี และยังมีลูกเล่นต่าง ๆให้ใช้ในการออกแบบอีกมากมาย และมีคอนเซปต์ในการเขียนอีก 3 คอนเซปต์ ดังนี้

1. Component คือ ส่วนต่าง ๆ ของหน้าเว็บ จะเหมือนกับการเขียน HTML แต่ React จะใช้สิ่งที่เรียกว่า JSX ในการแสดงผลเว็บไซต์ หน้าตาจะคล้ายๆ HTML แต่ต่างกันตรงจะใช้ JavaScript แทน

2. State คือ ข้อมูลที่อยู่ใน Component แต่ละชิ้น

3. Props ข้อมูลที่ถูกส่งต่อจาก Component ชั้นบนลงไปชั้นล่าง



รูปที่ 3.5 Concept หลักของ React
ที่มา (https://citly.me/Ly5H8)

โดยในโครงงานได้มีการใช้ React ในการสร้าง user interface ในส่วนการลงทะเบียนสมัครสมาชิก และการแสดงผลข้อมูลต่าง ๆ

3.9 Firebase [12]

Firebase คือ แพลตฟอร์มจาก Google ที่รวบรวมเครื่องมือต่าง ๆ สำหรับการจัดการในส่วนของ Backend หรือ Server side เช่น บริการทางด้านฐานข้อมูล บริการสำหรับการสร้างระบบล็อคอินและยืนยันตัวตน ช่วยลดเวลาและค่าใช้จ่ายของการทำ Server side หรือ การวิเคราะห์ข้อมูล

โดยในโครงงานนี้ได้มีการใช้ Firebase ในการเก็บข้อมูลต่าง ๆ ในระบบของผู้ใช้ อุปกรณ์ และข้อมูลของระบบแอดมิน โดยมีการใช้บริการ Firebase ดังนี้

1. Cloud Firestore เป็นบริการทางด้านฐานข้อมูลในรูปแบบ NoSQL ใช้สำหรับจัดเก็บข้อมูลและอัปเดตข้อมูลแบบเรียลไทม์

โดยนำบริการมาใช้ในการเก็บข้อมูลของระบบเว็บผู้ใช้ และระบบเว็บแอดมิน

2. Realtime Database เป็นบริการฐานข้อมูล ที่สามารถจัดเก็บข้อมูลและซิงค์ข้อมูลทั้งหมดแบบเรียลไทม์ และยังสามารถใช้งานร่วมกับอุปกรณ์ IoT

โดยนำบริการมาใช้ในการเก็บข้อมูลอุปกรณ์ และข้อมูลที่ถูกส่งเข้ามาจากอุปกรณ์ ESP32-CAM

3. Authentication เป็นบริการที่ใช้สำหรับตรวจสอบสิทธิ์ การเข้าถึงและยืนยันตัวตน

โดยนำบริการมาใช้ในการเก็บข้อมูลยืนยันตัวตนของระบบเว็บแอดมิน

3.10 โมเดล Random Forest Regression [13]

Random Forest Regression เป็นอัลกอริทึมการเรียนรู้ภายใต้การดูแลที่ใช้วิธีการเรียนรู้ทั้งมวลสำหรับการถดถอย วิธีการเรียนรู้แบบ Ensemble เป็นเทคนิคที่รวมการคาดการณ์จากอัลกอริทึมการเรียนรู้ของเครื่องหลายตัวเข้าด้วยกัน เพื่อการทำนายที่แม่นยำยิ่งขึ้นมากกว่าแบบจำลองเดียว

ขั้นตอนการสร้างโมเดล Random Forest Regression

1. เลือกจำนวนต้นแบบ (Number of Trees): ก่อนที่จะสร้าง Random Forest จำนวนต้นแบบ (Number of Trees) ต้องถูกกำหนดก่อน ซึ่งจำนวนต้นแบบนี้จะส่งผลต่อประสิทธิภาพและเวลาที่ใช้ในการทำนาย จำนวนต้นแบบที่ควรใช้จะเป็นจำนวนที่เหมาะสมกับข้อมูลและการทำงานที่ต้องการ ซึ่งปกติจะใช้จำนวนต้นแบบที่มากพอที่จะลดความคลาดเคลื่อนและเพิ่มความแม่นยำ

2. Bootstrap Sampling: ขั้นตอนแรกในการสร้าง Random Forest คือการสุ่มชุดข้อมูลจากชุดข้อมูลที่มีอยู่โดยใช้เทคนิค Bootstrap Sampling ที่ทำให้สุ่มข้อมูลมาโดยการสุ่มแบบถอดกลับ (with replacement) หรือสุ่มตัวอย่างที่มีขนาดเท่ากับจำนวนข้อมูลทั้งหมด แต่ข้อมูลบางส่วนอาจถูกเลือกมาหลายครั้ง การสุ่มแบบนี้ช่วยให้ความหลากหลายของต้นแบบและลดความเสี่ยงในการเกิดการเรียนรู้เกินอาจารย์ (Overfitting)

3. สร้างต้นแบบ (Decision Trees): หลังจากที่ได้ข้อมูลที่ถูกสุ่มมาแล้ว ก็จะเริ่มสร้างต้นแบบ Decision Trees สำหรับแต่ละต้นแบบ โดยในแต่ละโหนด (node) ของต้นแบบ Decision Trees จะทำการสุ่มคุณสมบัติบางส่วนเพื่อใช้ในการตัดสินใจ โดยเปรียบเทียบค่าของคุณสมบัติที่ถูกสุ่มมากับค่า threshold ที่กำหนดเพื่อแยกข้อมูลออกเป็นกลุ่มย่อย ๆ ตามคุณสมบัติที่ถูกสุ่ม

ขั้นตอนการทำนายผลของโมเดล Random Forest Regression หลังจากที่ทำการสร้างโมเดลแล้ว

1. รับข้อมูลนำเข้า นำข้อมูลที่ต้องการทำนายเข้าสู่โมเดล

2. การทำนาย โมเดล Random Forest Regression จะใช้กฎการตัดสินใจจากแต่ละต้นไม้ เพื่อทำนายผล ต้นไม้แต่ละต้นจะให้ผลลัพธ์ตามกฎการตัดสินใจของตนเอง.

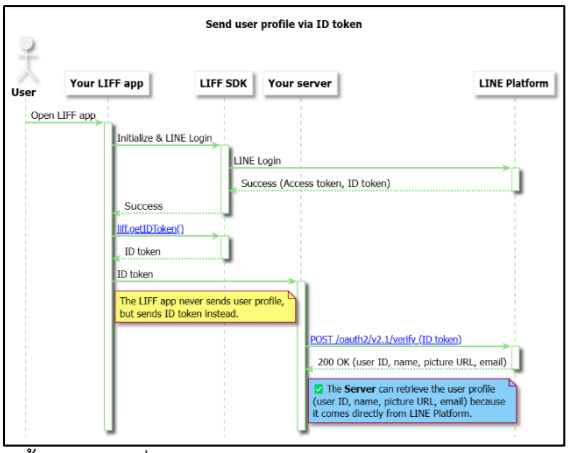
3. รวมผลลัพธ์ ผลลัพธ์ที่ได้จากแต่ละต้นไม้จะรวมเข้าด้วยกัน โดยการหาค่าเฉลี่ย เพื่อให้ได้ผลลัพธ์สุดท้าย

โครงงานนี้ใช้โมเดล Random Forest Regression ในการทำนายอุณหภูมิของสายไฟว่าเสี่ยงที่จะเกิดเหตุเพลิงไหม้หรือไม่ เนื่องจากต้องการทำนายค่าตัวแปรที่เป็นตัวเลขและมีความซับซ้อนของข้อมูล โมเดลนี้สามารถทำนายข้อมูลที่ต้องการความแม่นยำสูง และเหมาะกับข้อมูลที่มีความซับซ้อน โมเดล Random Forest Regression มีการใช้ต้นไม้หลายต้นเพื่อช่วยในการตัดสินใจซึ่งจะช่วยลดปัญหาที่เกิดจากการ overfitting และช่วยให้การทำนายมีความแม่นยำขึ้น โดยใช้ไลบรารี Random Forest Regression ใน sklearn เพื่อสร้างโมเดลทำนายผลออกมาดังขั้นตอนการสร้างโมเดล Random Forest ที่ได้มีการอธิบายก่อนหน้า

3.11 LINE Front-end Framework (LIFF) And LINE Login [14]

เป็นเฟรมเวิร์คที่พัฒนาโดย LINE Corporation เพื่อช่วยให้นักพัฒนาสามารถสร้างแอปพลิเคชันบนแพลตฟอร์ม LINE ได้ง่ายขึ้น โดย LIFF ให้ความสามารถในการนำเสนอแอปพลิเคชันภายใน LINE ที่เปิดโอกาสในการเชื่อมต่อกับข้อมูลและบริการต่าง ๆ ใน LINE อย่างที่ไม่ต้องออกนอกแอป LINE หลัก

ด้วย LIFF นักพัฒนาจึงสามารถเขียนแอปพลิเคชันเพื่อใช้งานหลากหลายฟีเจอร์ และการเชื่อมต่อกับบริการต่าง ๆ ของ LINE เช่น การเข้าสู่ระบบ การเข้าถึงข้อมูลโปรไฟล์ของผู้ใช้ เพื่อนำข้อมูลของผู้ใช้ไปใช้งานในระบบ และบริการอื่น ๆ ต่อไป



รูปที่ 3.6 ขั้นตอนการที่จะนำข้อมูลผู้ใช้มาใช้งานในเซิร์ฟเวอร์ของนักพัฒนา

(Using user data in LIFF apps and servers | LINE Developers)

โดยในโครงงานวิจัยนี้ได้มีการใช้ LIFF ในกระบวนการเข้าสู่ระบบและสมัครสมาชิกผ่าน LINE เพื่อให้สะดวกกับผู้ใช้ในการใช้งานเว็บแอปพลิเคชัน

บทที่ 4 ขั้นตอนและแผนการดำเนินงาน

ในบทนี้จะประกอบไปด้วยขั้นตอนการดำเนินงาน เวลาดำเนินการทำงาน และรายละเอียดของการดำเนินงานในแต่ละขั้นตอนโดยจะกล่าวถึงแผนการดำเนินงาน

4.1 ขั้นตอนการดำเนินงาน

4.1.1 คิดหัวข้อและเสนอหัวข้อโครงงาน

- นำสิ่งที่สนใจมานำเสนอโครงงานกับอาจารย์ที่ปรึกษา
- เลือกหัวข้อโครงงาน

4.1.2 ศึกษารวบรวมข้อมูลและกำหนดขอบเขตโครงงาน

- รวบรวมข้อมูลที่เกี่ยวข้องกับโครงงาน รวมถึงการวิเคราะห์ข้อมูลที่เป็นไปได้เพื่อออกแบบระบบงาน
- กำหนดขอบเขตโครงงาน เพื่อเป็นเกณฑ์ในการทำงาน

4.1.3 ศึกษางานที่เกี่ยวข้องกับโครงงาน

- ศึกษางานวิจัยหรือผลงานที่มีความคล้ายคลึงกับระบบงาน และนำมาปรับใช้ให้เหมาะสมกับงาน

4.1.4 ศึกษาความรู้และทฤษฎีที่เกี่ยวข้อง

- ศึกษาอุปกรณ์ฮาร์ดแวร์และซอฟต์แวร์ที่ต้องใช้ในโครงงาน

4.1.5 ศึกษาการนำข้อมูลมาใช้ในงานซอฟต์แวร์

- ศึกษาการทำระบบ IoT และการนำข้อมูลจากฮาร์ดแวร์มาแสดงบนหน้าเว็บ

4.1.6 วิเคราะห์และออกแบบระบบ

- วิเคราะห์และออกแบบระบบให้เหมาะสมกับงาน

4.1.7 พัฒนาระบบ

- ทำระบบตรวจวัดอุณหภูมิและตรวจจับภาพ โดยเขียนคำสั่งลงตัวคอนโทรลเลอร์ และทำให้ตัวคอนโทรลเลอร์ส่งข้อมูลมาให้ Server เพื่อแสดงผลให้ User ได้เห็น

4.1.8 ทดสอบและปรับปรุงการทำงานของระบบ

- ทดสอบว่าระบบที่พัฒนาขึ้นตรงตามขอบเขตที่ได้กำหนดไว้หรือไม่ และปรับปรุงแก้ไขในส่วนที่ไม่ตรงตามขอบเขตที่กำหนด

4.1.9 จัดทำเอกสารประกอบโครงงาน

- จัดทำเอกสารรายละเอียดของโครงงานตั้งแต่ขั้นตอนเริ่มต้นไปจนถึงขั้นตอนสุดท้ายของการทำโครงงาน

4.2 แผนการดำเนินงาน

ตารางแสดงระยะในการดำเนินงานในแต่ละขั้นตอน

ตารางที่ 4.1 ขั้นตอนและแผนการดำเนินงาน

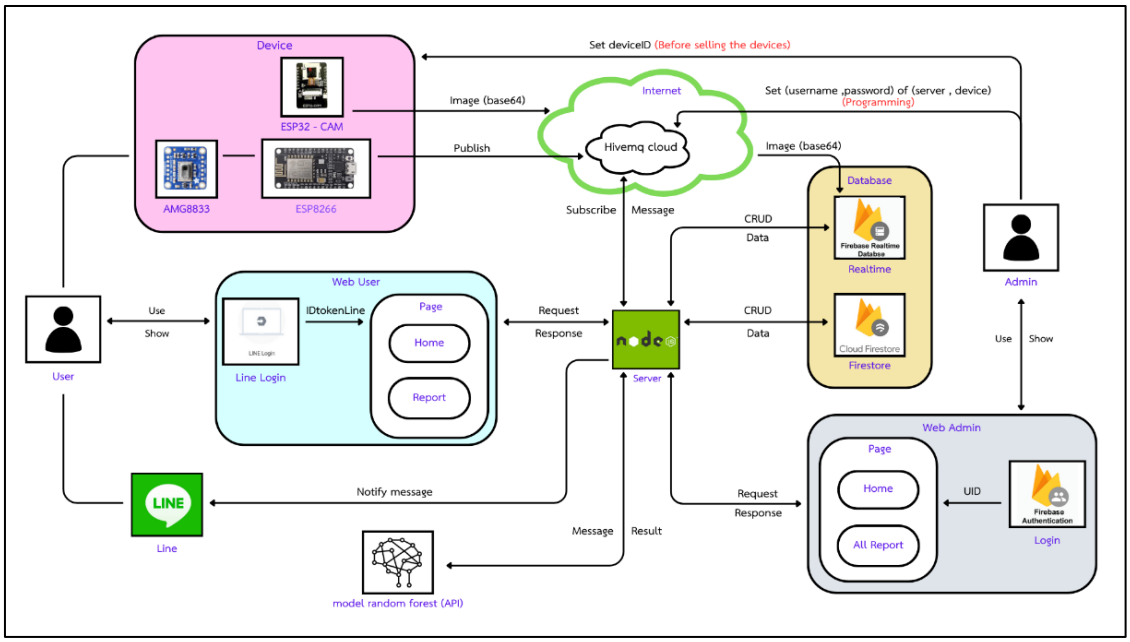
ขั้นตอนการดำเนินงาน	ก.ค. 2565	ส.ค. 2565	ก.ย. 2565	-	ธ.ค. 2565	ม.ค. 2566	-	ก.ค. 2566	ส.ค. 2566	ก.ย. 2566	ต.ค. 2566
1. เสนอหัวข้อโครงงาน	↔	↔	↔	→							
2. ศึกษารวบรวมข้อมูลและกำหนดของเขตของโครงงาน	↔	↔	↔	↔	→						
3. ศึกษางานที่เกี่ยวข้องกับโครงงาน		↔	↔	↔	↔	→					
4. ศึกษาความรู้และทฤษฎีที่เกี่ยวข้อง		↔	↔	↔	↔	→					
5. ศึกษาการนำข้อมูลมาใช้ในงานซอฟต์แวร์		↔	↔	↔	↔	→					
6. วิเคราะห์และออกแบบระบบ	↔	↔	↔	↔	↔	↔	→				
7. พัฒนาระบบ			↔	↔	↔	↔	↔	↔	↔	↔	→
8. ทดสอบและปรับปรุงการทำงานของระบบ			↔	↔	↔	↔	↔	↔	↔	↔	→
9. จัดทำเอกสารประกอบโครงงาน			↔	↔	↔	↔	↔	↔	↔	↔	→

บทที่ 5 การวิเคราะห์และออกแบบระบบ

ในบทนี้จะกล่าวถึงกระบวนการดำเนินงานของระบบ โครงสร้างการทำงานของระบบ โดยจะแสดงโครงสร้างภาพรวม โครงสร้างแต่ละส่วนของระบบ และรูปแบบแอปพลิเคชัน

5.1 โครงสร้างการทำงานของระบบ

การสร้างระบบการแจ้งเตือนการเกิดเหตุเพลิงไหม้ผ่านทางไลน์ โดยมีวัตถุประสงค์เพื่อให้ผู้ใช้สามารถแก้ไขปัญหาได้ทันก่อนเกิดเหตุเพลิงไหม้รุนแรง



รูปที่ 5.1 โครงสร้างการทำงานของระบบ

จากรูปที่ 5.1 จะเห็นได้ว่าโครงสร้างของระบบจะมีส่วนประกอบต่าง ๆ ดังนี้

1) ส่วนของอุปกรณ์จะแยกเป็น 2 ส่วน

- ส่วนที่หนึ่ง คือ ส่วนที่ใช้ในการตรวจจับค่าอุณหภูมิความร้อนจะใช้เซนเซอร์ AMG8833 ต่อเข้ากับบอร์ด ESP8266 และทำการเชื่อมต่ออินเทอเน็ต ตัวอุปกรณ์จะมีหน้าที่เป็น Publish และกำหนด Topic เพื่อส่งข้อมูลค่าอุณหภูมิที่ตรวจจับได้ไปยัง MQTT Broker ทุกๆ 10 วินาที หลังจากนั้นจะส่งข้อมูลต่อไปให้ Node JS เพื่อใช้ในการวิเคราะห์ต่อไป

- ส่วนที่สอง คือ ส่วนของ ESP32-CAM จะทำการจับภาพทุกๆ 30 วินาทีและทำการส่งภาพไปที่ firebase ในรูปแบบ base64

2) Node.JS ทำการเรียกใช้ โมดูล express.js ใช้สำหรับการทำงานในส่วนเซิร์ฟเวอร์ (Back-end) หน้าที่ของเซิร์ฟเวอร์มีดังนี้

- สร้างส่วนติดต่อกับหน้าเว็บไซต์ (front-end) ที่เรียกว่า API Endpoint ที่จะทำการรับคำของจากหน้าเว็บไซต์มาวิเคราะห์ และสงผลลัพธ์จากการวิเคราะห์กลับไปยังหน้าเว็บไซต์เพื่อใช้ในการแสดงผลต่อไป

- เชื่อมต่อกับ HiveMQ (MQTT broker) ทำการสมัครรับข้อมมูล (Subscribe) จากตัวอุปกรณ์ทั้งหมดที่มีอยู่ในฐานข้อมูล รับข้อมูลอุณหภูมิที่ส่งมาจากแต่ละอุปกรณ์ เพื่อทำการวิเคราะห์และส่งแจ้งเตือนไปยังผู้ใช้งาน

- เชื่อมต่อกับฐานข้อมูล Firebase เพื่อใช้ในการเพิ่ม เรียกดู แก้ไข และลบข้อมูล จากคำข้อที่ถูกส่งเข้ามาจากส่วนหน้า

- เชื่อมต่อกับ API และส่งชุดอุณหภูมิที่ได้รับจากอุปกรณ์ไปทำนายผล

3) Firebase ทำหน้าที่เป็นฐานข้อมูลเก็บข้อมูลของผู้ใช้ รหัสประจำตัวผู้ใช้งาน ชื่อของผู้ใช้งาน ชื่อบ้านผู้ใช้งาน เก็บชื่อห้องทั้งหมดภายในบ้านของผู้ใช้ และรหัสอุปกรณ์ที่ติดตั้งประจำแต่ละห้องของผู้ใช้

4) React.js ใช้ทำเว็บแอปพลิเคชัน

- เว็บแอปพลิเคชันผู้ใช้ สมัครสมาชิกและเข้าสู่ระบบเพื่อให้ผู้ใช้ดูข้อมูลต่าง ๆ ทำการเพิ่มห้องและอุปกรณ์ที่ติดตั้ง แก้ไขข้อมูลชื่อบ้าน ชื่อห้องและรหัสอุปกรณ์ ลบห้องที่ติดตั้งอุปกรณ์ และแจ้งปัญหาของอุปกรณ์ในกรณีที่อุปกรณ์มีปัญหา

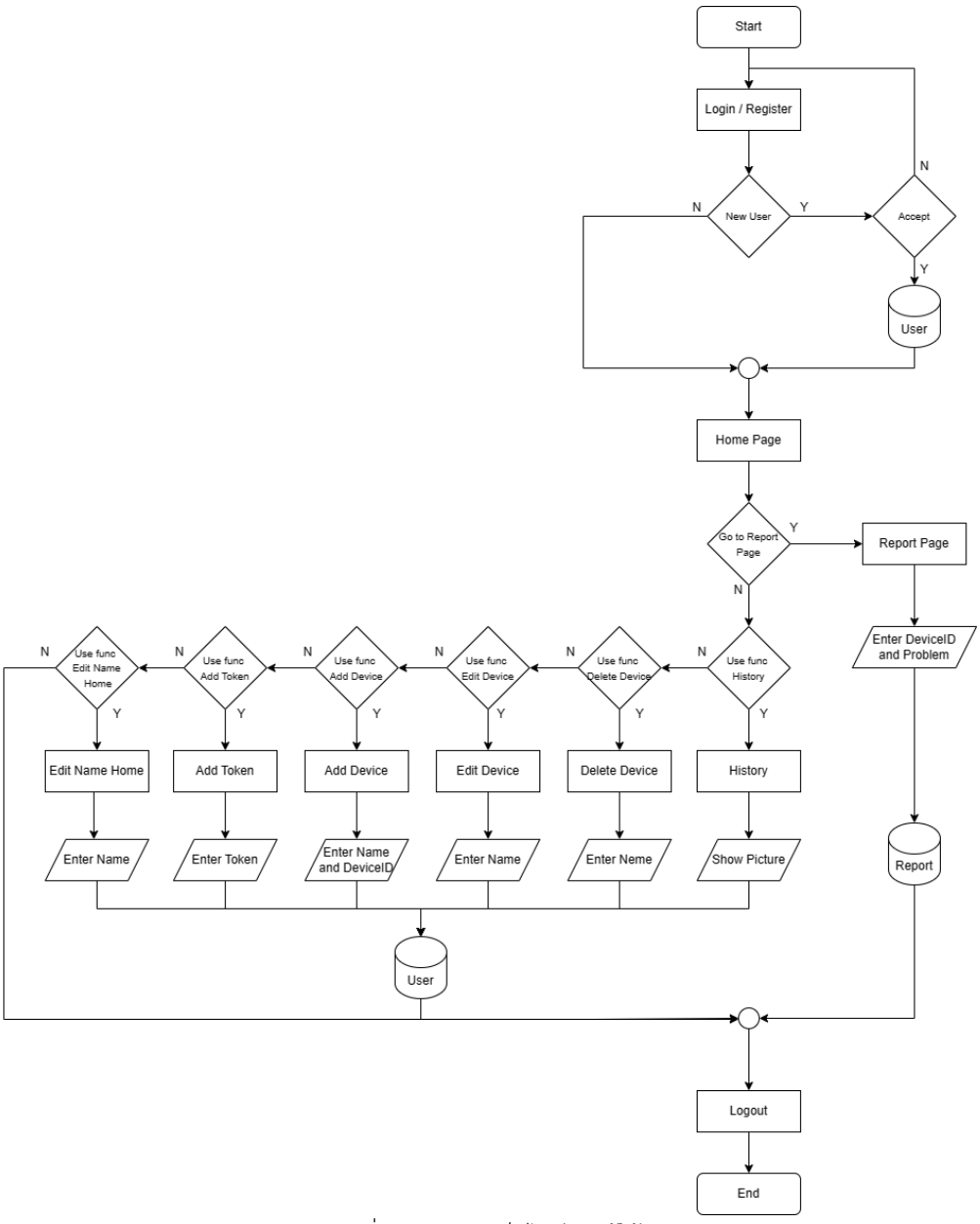
- เว็บแอปพลิเคชันของผู้ดูแลระบบ ใช้สำหรับดูข้อมูลอุปกรณ์ทั้งหมด และเพิ่มอุปกรณ์ใหม่เข้าระบบ ใช้ดูรายงานความผิดปกติของอุปกรณ์เมื่อผู้ใช้แจ้งเข้ามา

5) Model Random Forest (API) ใช้เพื่อเรียกใช้ตัวโมเดลและวิเคราะห์ค่าอุณหภูมิที่ได้ว่าเป็นเหตุการที่มีความร้อนสูงกว่าปกติหรือไม่ และส่งผลลัพธ์กลับไปยัง Server

5.2 แผนผังการทำงานโดยรวมของระบบ (Flow Chart Diagram)

5.2.1 เว็บไซต์ของระบบ

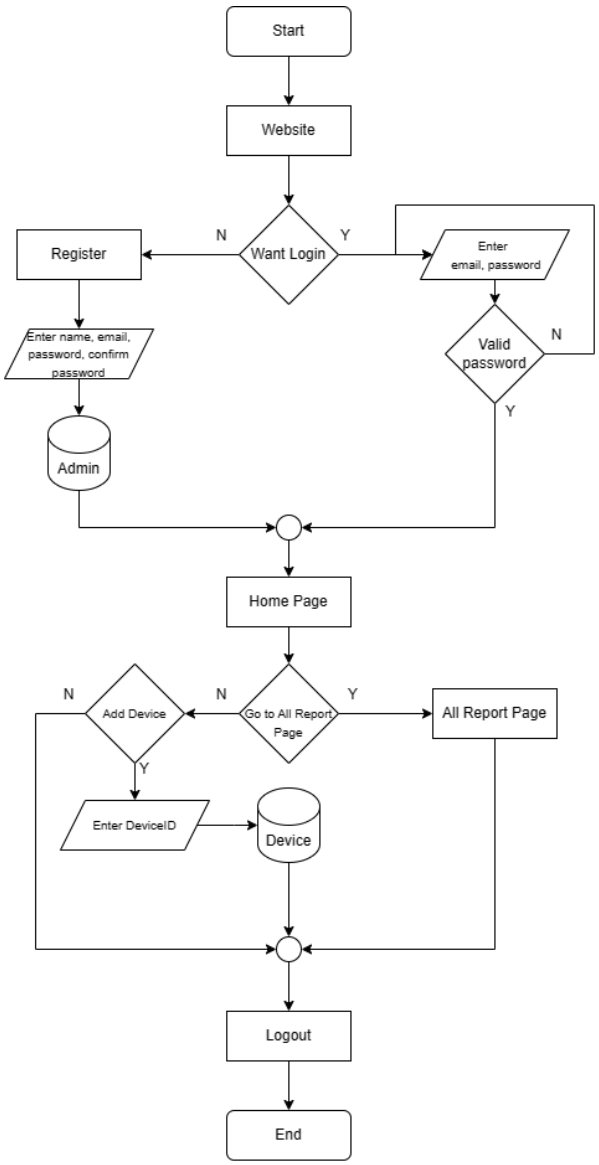
1) เว็บไซต์ของผู้ใช้



รูปที่ 5.2 แผนผังเว็บไซต์ของผู้ใช้

จากรูปที่ 5.2 กระบวนการทำงานของผู้ใช้ จะเริ่มต้นจากให้ผู้ใช้ทำการล็อกอิน ในกรณีที่เป็นผู้ใช้ใหม่จะต้องทำการกดอนุญาติการเข้าถึงจากไลน์ก่อน เมื่อทำการกดอนุญาตเสร็จข้อมูลจะถูกจัดเก็บลงในฐานข้อมูลผู้ใช้ หลังจากเข้าล็อกอินเข้าใช้งานเรียบร้อยแล้ว ในเว็บจะมี 2 หน้าในการทำงาน คือ หน้าหลัก และ หน้ารายงานปัญหา หน้าหลักจะใช้สำหรับ จัดการบ้านและอุปกรณ์ ส่วนหน้ารายงานปัญหาใช้สำหรับให้ผู้ใช้รายงานปัญหาที่พบ และจัดเก็บไว้ในฐานข้อมูลของผู้ใช้และฐานข้อมูลของรายงานตามลำดับ

2) เว็บไซต์ของผู้ดูแลระบบ



รูปที่ 5.3 แผนผังเว็บไซต์ของผู้ดูแลระบบ

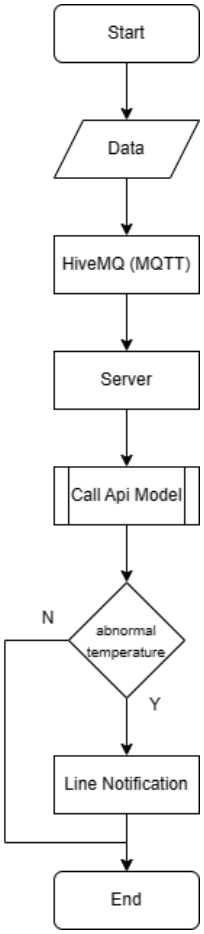
จากรูปที่ 5.3 กระบวนการทำงานของผู้ดูแลระบบ เริ่มต้นจากการเข้าสู่เว็บไซต์ของผู้ดูแลระบบ มี 2 กรณี

- กรณีที่ 1 เป็นผู้ดูแลระบบอยู่แล้ว ทำการล็อกอิน โดยกรอก e-mail, password หากกรอก email, password ผิดจะต้องทำกรอกใหม่
- กรณีที่ 2 เป็นผู้ดูแลระบบใหม่ ต้องกรอกชื่อ, e-mail, password และยืนยัน password เพื่อจัดเก็บข้อมูลเข้าฐานข้อมูลของผู้ดูแลระบบ

หลังจากที่เข้าสู่เว็บไซต์สำเร็จจะพบหน้าหลักของเว็บไซต์ซึ่งหน้าหลักจะมีการแสดงข้อมูลอุปกรณ์ทั้งหมดในระบบ และมีฟีเจอร์การเพิ่มอุปกรณ์โดยกรอก DeviceID เพื่อจัดเก็บเข้าฐานข้อมูลของอุปกรณ์ ผู้ดูแลระบบสามารถดูรายงานปัญญาทั้งหมดจากผู้ใช้ที่หน้า All Report และสามารถล็อกเอาท์ออกจากเว็บไซต์ จบการทำงาน

5.2.2 การทำงานของอุปกรณ์และการแจ้งเตือนไลน์

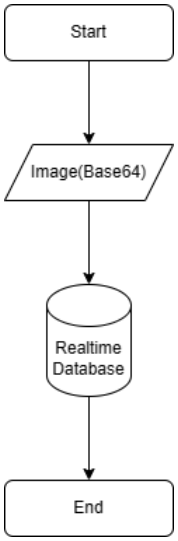
1) การทำงานของเซนเซอร์ AMG8833 ส่งข้อมูลไปยัง Server เพื่อทำนายผลและส่งการแจ้งเตือน



รูปที่ 5.4 แผนผังการทำงานของอุปกรณ์ AMG8833 และการแจ้งเตือนไลน์

จากรูปที่ 5.4 การทำงานของ AMG8833 และการแจ้งเตือนไลน์ เริ่มการทำงาน AMG8833 ที่เชื่อมต่ออยู่กับ ESP8266 จะส่งข้อมูลอุณหภูมิไปที่ HiveMQ จากนั้น Sever จะรับข้อมูลอุณหภูมิที่ถูกส่งมาจาก HiveMQ และส่งข้อมูลอุณหภูมิไปให้ API Model ทำนายผลว่าข้อมูลอุณหภูมินั้นมีความผิดปกติหรือไม่ หากมีความผิดปกติจะทำการส่งแจ้งเตือนไลน์ไปยังผู้ใช้ หากข้อมูลอุณหภูมิไม่มีความผิดปกติจะจบการทำงาน

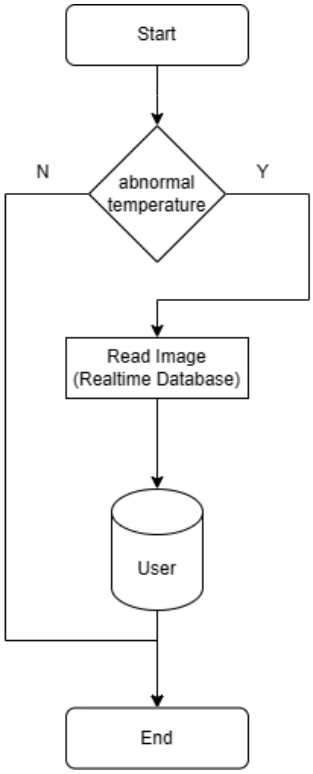
2) การทำงานของ ESP32-CAM ส่งข้อมูลไปที่ Realtime Database



รูปที่ 5.5 แผนผังการทำงานของอุปกรณ์ ESP32-CAM

จากรูปที่ 5.5 การทำงานของ ESP32-CAM จะเริ่มทำงการจับภาพทุกๆ 3 วินาที หลังจากนั้นจะส่งภาพที่จับได้ในรูปแบบ base64 ไปจัดเก็บไว้ที่ Realtime Database จบการทำงานในส่วนของ ESP32-CAM

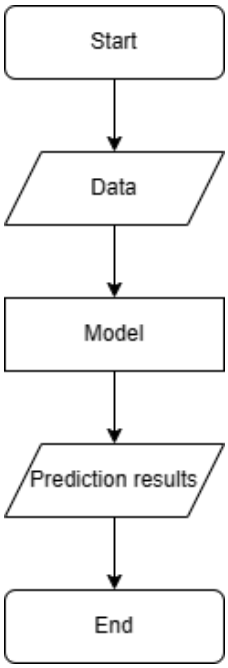
3) การบันทึกภาพที่ได้จาก ESP32-CAM ไปยังฐานข้อมูลของผู้ใช้หลังจากทำนายว่าเกิดเหตุขึ้นแล้ว



รูปที่ 5.6 แผนผังการทำงานการบันทึกภาพไปยังฐานข้อมูลของผู้ใช้

จากรูปที่ 5.6 การบันทึกภาพไปยังฐานข้อมูลของผู้ใช้การทำงานจะเริ่มหลังจากที่ Server รับผลทำนายจาก API Model ว่าอุณหภูมิมีความผิดปกติ หลังจากนั้นจะนำภาพที่จัดเก็บอยู่ใน Realtime Database ในขณะที่อุณหภูมิมีความผิดปกติไปจัดเก็บไว้ที่ฐานข้อมูลของผู้ใช้ แต่ถ้า API Model ทำนายว่าอุณหภูมิปกติ จะจบการทำงาน

5.2.3 การทำงานของโมเดล (Random Forest)



รูปที่ 5.7 แผนผังการทำงานของโมเดล

จากรูปที่ 5.7 การทำงานของโมเดล เริ่มต้นโดยการรับข้อมูลอุณหภูมิและเรียกใช้โมเดลเพื่อทำนายผล จากนั้นจะแสดงผลลัพธ์การทำนาย จบการทำงานของโมเดล

5.3 รูปแบบเว็บไซต์

5.3.1 เว็บไซต์ในส่วนของผู้ใช้

1) หน้าเริ่มต้นของเว็บไซต์ของผู้ใช้ เป็นหน้าแรกของเว็บไซต์ก่อนเริ่มต้นใช้งาน และมีปุ่มเข้าสู่ระบบ (Login)

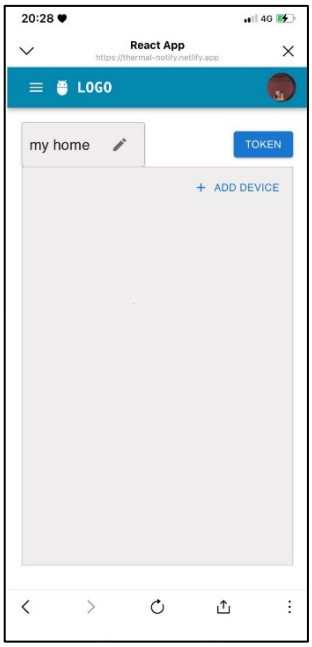
รูปที่ 5.8 หน้าเริ่มต้นของเว็บไซต์ของผู้ใช้

2) หน้าสมัครสมาชิก เป็นหน้าการสมัครการใช้งานของเว็บไซต์โดยล็อกอินไลน์

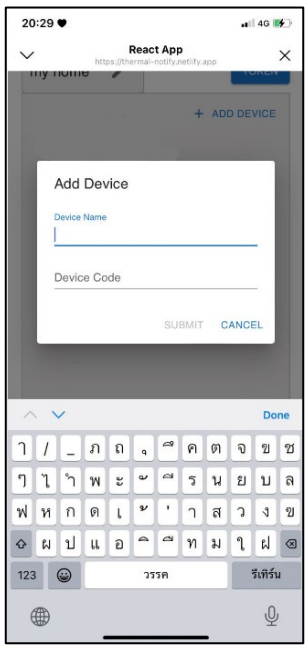
รูปที่ 5.9 หน้าล็อกอินไลน์

รูปที่ 5.10 หน้าอนุญาติสิทธิ์การเข้าถึง

3) หน้าหลักของเว็บไซต์ เป็นหน้าแรกหลังจากผู้ใช้ได้ทำการล็อกอินเข้ามาเมื่อผู้ใช้กดปุ่มเพิ่มอุปกรณ์ (Add Device) จะขึ้นดังรูปที่ 5.8

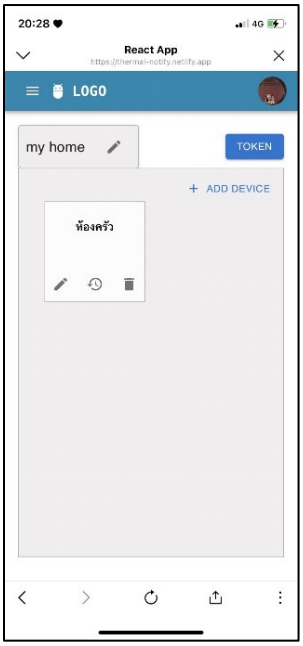


รูปที่ 5.11 หน้าหลักของเว็บไซต์ของผู้ใช้



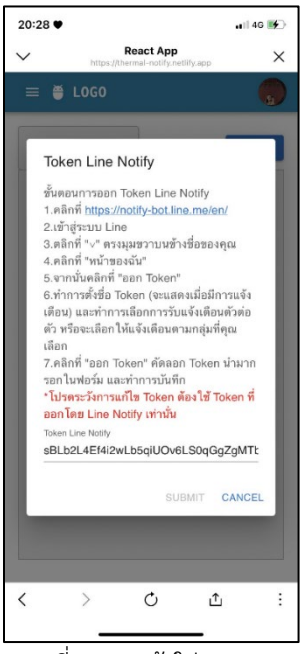
รูปที่ 5.12 หน้าเพิ่มอุปกรณ์ส่วนของผู้ใช้

และเมื่อผู้ใช้กดปุ่มยืนยัน (submit) จะปรากฏผลลัพธ์ดังรูปที่ 5.13



รูปที่ 5.13 หน้าหลักของเว็บไซต์ของผู้ใช้หลังมีการเพิ่มอุปกรณ์

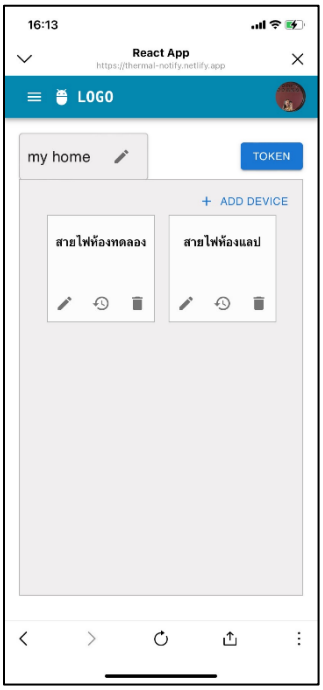
4) หน้าใส่ Token เป็นหน้าที่นำ Token ที่ผู้ใช้ได้ทำการออก มาใส่โดยกดที่ปุ่ม TOKEN จากรูปที่ 5.14



รูปที่ 5.14 หน้าใส่ Token

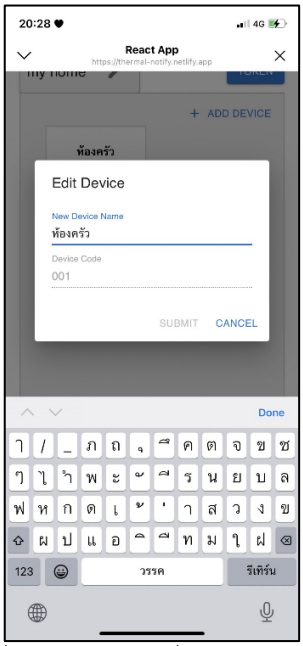
5) หน้าแก้ไขชื่อ เป็นหน้าที่ผู้ใช้สามารถแก้ไขชื่ออุปกรณ์ได้ตามที่ผู้ใช้ต้องการ

5.1) หน้าแก้ไขชื่อบ้าน ผู้ใช้กดปุ่มไอคอนรูปดินสอจากรูป 5.15



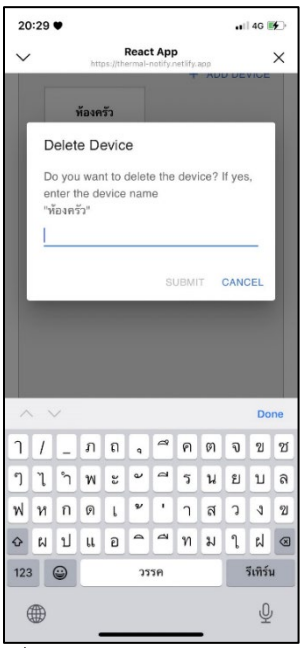
รูปที่ 5.15 หน้าแก้ไขชื่อบ้านของผู้ใช้

5.2) หน้าแก้ไขชื่ออุปกรณ์ ผู้ใช้กดปุ่มรูปไอคอนดินสอที่กล่องอุปกรณ์ที่ต้องการจะแก้ไข



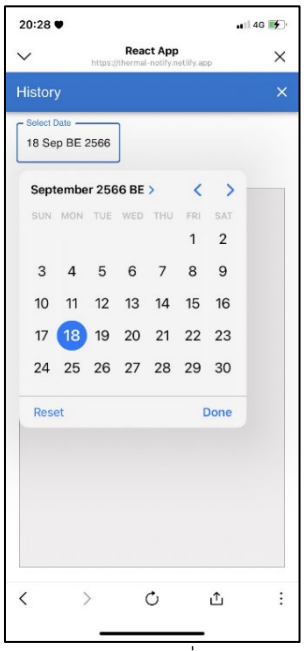
รูปที่ 5.16 หน้าแก้ไขชื่ออุปกรณ์ของผู้ใช้

6) หน้าลบอุปกรณ์ เป็นหน้าที่ผู้ใช้สามารถลบอุปกรณ์ที่ไม่ต้องการได้ ทำได้โดยการลยปุ่มไอคอนรูปถังขยะ

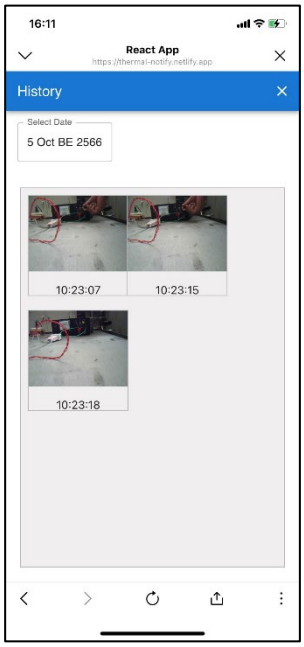


รูปที่ 5.17 หน้าลบอุปกรณ์ของผู้ใช้

7) หน้าดูประวัติการเกิดเหตุ เป็นหน้าที่ผู้ใช้สามารถเข้ามาดูภาพประวัติการเกิดเหตุได้ สามารถเลือกวันที่ที่เกิดเหตุได้ โดยทำการดูได้จากการกดปุ่มไอคอนรูปนาฬิกา

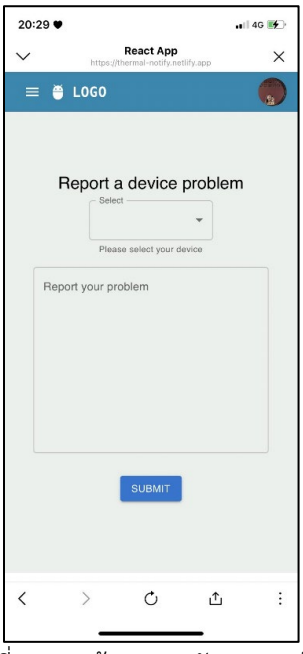


รูปที่ 5.18 หน้าเลือกวันที่ดูประวัติของผู้ใช้



รูปที่ 5.19 หน้าดูรูปประวัติการเกิดเหตุ

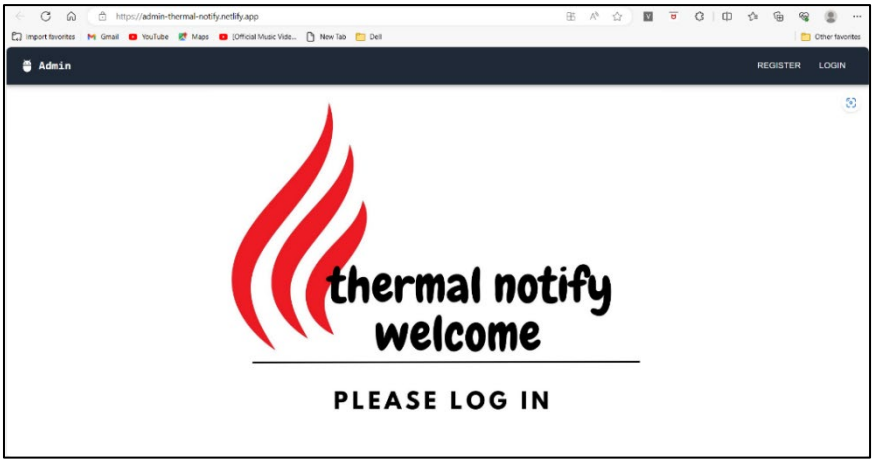
8) หน้ารายงานปัญหา เป็นหน้าที่ผู้ใช้จะทำการรายงานปัญหาต่าง ๆที่เกิดขึ้น



รูปที่ 5.20 หน้ารายงานปัญหาของผู้ใช้

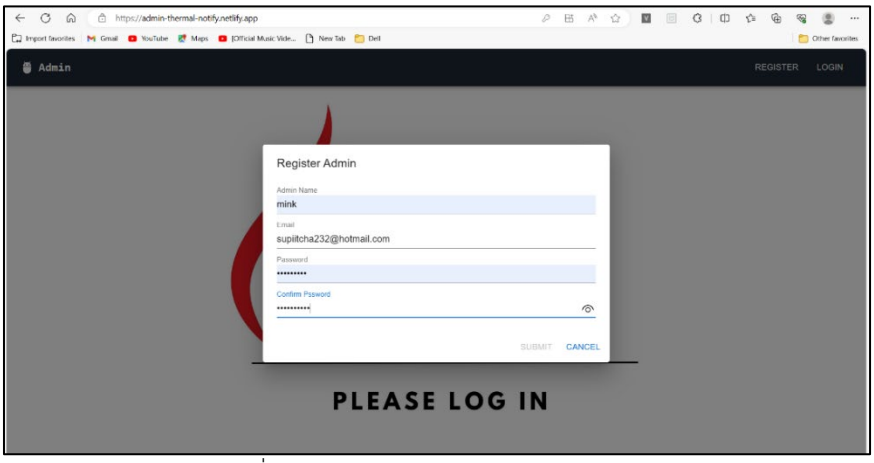
5.3.2 เว็บไซต์ในส่วนของผู้ดูแลระบบ

1) หน้าแรกเว็บไซต์ของผู้ดูแลระบบ เป็นหน้าเริ่มการเข้าใช้งาน



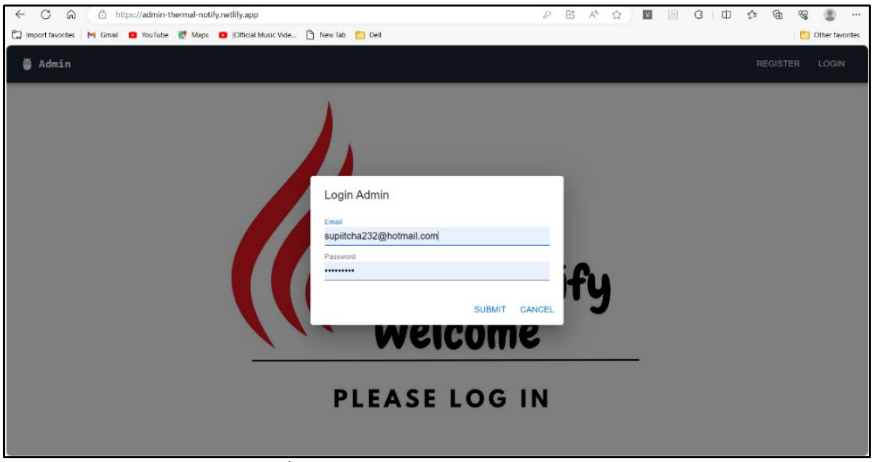
รูปที่ 5.21 หน้าแรกเว็บไซต์ของผู้ดูแลระบบ

ในกรณีที่ยังไม่เคยเป็นสมาชิกผู้ดูแลระบบจะต้องทำการสมัครสมาชิกโดยกดที่ปุ่ม Register ในรูป 5.21



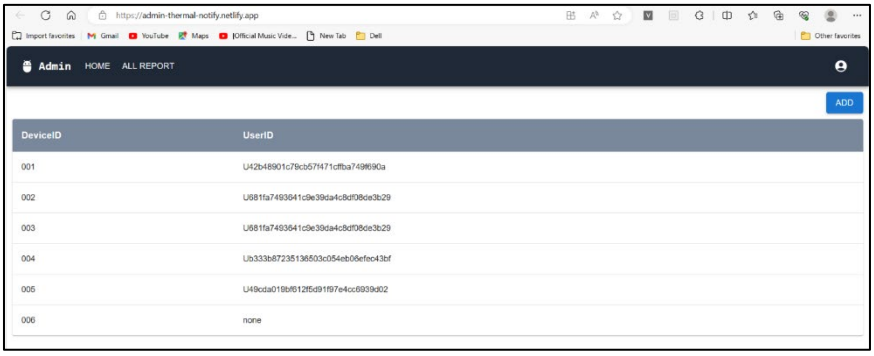
รูปที่ 5.22 หน้าสมัครสมาชิกของผู้ดูแลระบบ

2) หน้าล็อกอินของผู้ดูแลระบบ



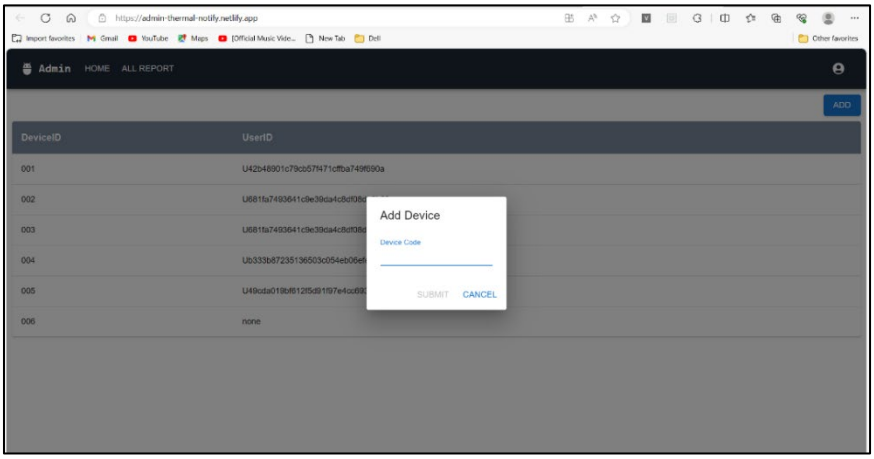
รูปที่ 5.23 หน้าล็อกอินของผู้ดูแลระบบ

3) หน้าหลักเว็บไซต์ของผู้ดูแลระบบ



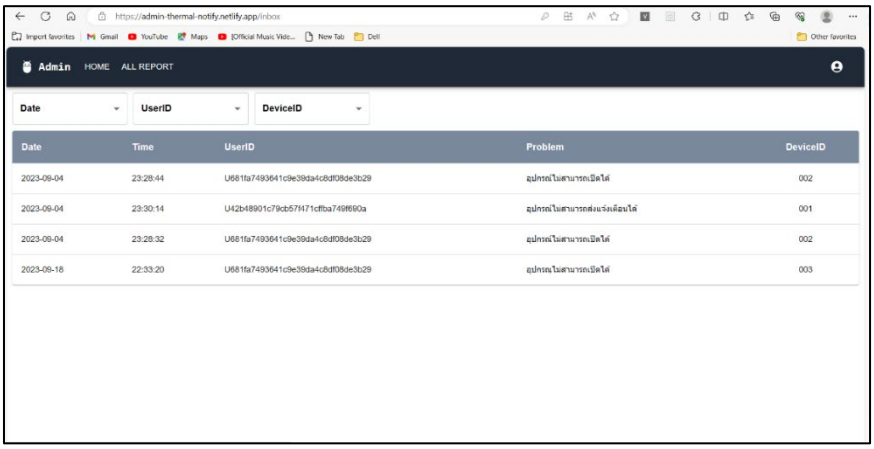
รูปที่ 5.24 หน้าหลักเว็บไซต์ของผู้ดูแลระบบ

4) หน้าเพิ่มอุปกรณ์ของผู้ดูแลระบบ สามารถทำการเพิ่มอุปกรณ์ได้หลังจากที่กดปุ่ม ADD ที่หน้าหลักจาก รูป 5.24



รูปที่ 5.25 หน้าเพิ่มอุปกรณ์ของผู้ดูแลระบบ

5) หน้ารายงานปัญหาของผู้ดูแลระบบ สามารถเลือก วันที่, ID ผู้ใช้ และรหัสของอุปกรณ์ได้



รูปที่ 5.26 หน้ารายงานปัญหาของผู้ดูแลระบบ

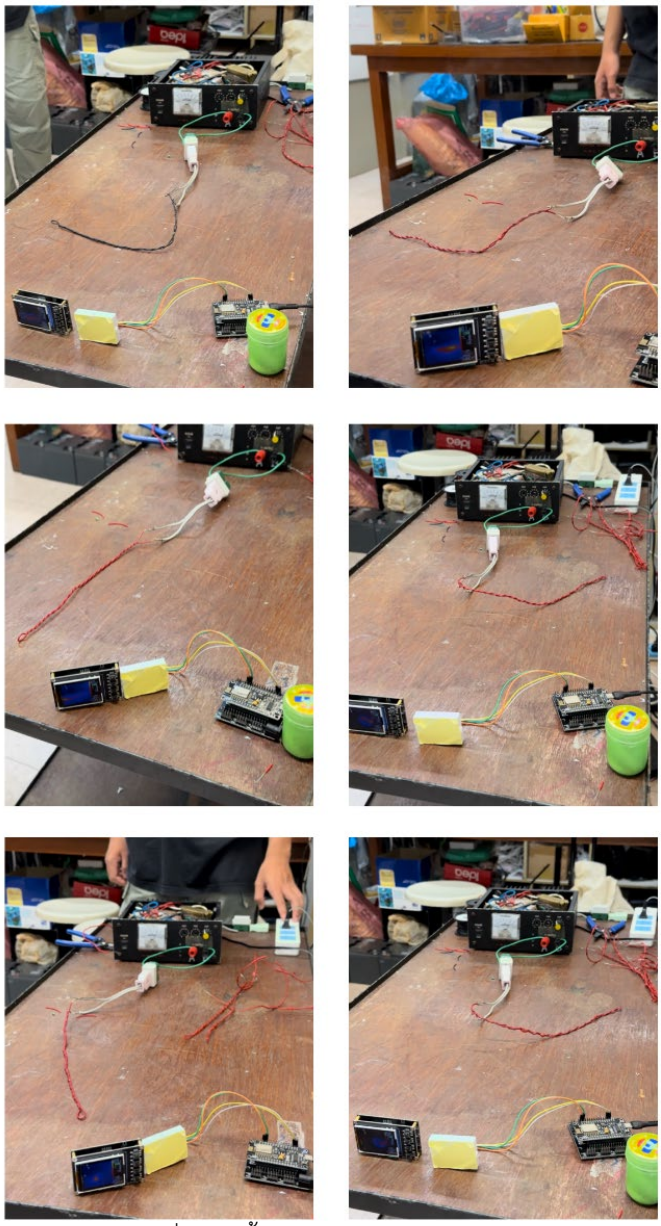
5.4 การทำการทดลองเก็บข้อมูลสำหรับการทดลองโมเดล (Random Forest)

5.4.1 ขั้นตอนการทดลองเก็บข้อมูล

ทำการทดลองเก็บข้อมูล ดังรูปที่ 5.24 โดยทำการทดลองเก็บข้อมูลจำนวน 11 ครั้ง ซึ่งในแต่ละครั้งจะมีตำแหน่งแตกต่างกันเพื่อให้ได้ข้อมูลอุณหภูมิที่หลากหลาย โดยระยะห่างระหว่างสายไฟและเซนเซอร์ AMG8833 ต้องไม่เกิน 30 เซนติเมตร

อุปกรณ์ที่ใช้ในการทดลองเก็บข้อมูล

1) Power supply ทำหน้าที่เป็นปลั๊กไฟ
2) สายไฟ UL Style No.1007 CSA Type TR-64 ความยาว 60 เซนติเมตรที่ม้วนเป็นเกลียว
3) เซนเซอร์ AMG8833 เชื่อมต่อกับ บอร์ด ESP8266



รูปที่ 5.27 ขั้นตอนการทดลองเก็บข้อมูล

จำนวนชุดข้อมูลอุณหภูมิที่ได้ในแต่ละครั้งของการทดลองเก็บข้อมูล เป็นดังตารางที่ 5.1

ครั้งที่	จำนวนชุดข้อมูลที่ได้
1	276
2	246
3	646
4	255
5	384
6	341
7	605
8	413
9	466
10	157
11	577
รวม	4366

ตารางที่ 5.1จำนวนชุดข้อมูลที่ได้ในแต่ละครั้ง

หลังจากที่ทำการทดลองเก็บข้อมูล จำนวนชุดข้อมูลอุณหภูมิที่ได้ทั้งหมดคือ 4366 ชุด

5.4.2 ค่าอุณหภูมิที่ได้จากการทดลองเก็บข้อมูล

จากการทดลองเก็บข้อมูลได้มีการอัดคลิปวิดีโอเพื่อดูความเปลี่ยนแปลงของอุณหภูมิของสายไฟตั้งแต่ก่อนจะมีการจ่ายกระแสไฟฟ้าจนกระทั่งสายไฟละลาย จากการทดลองทั้งหมด 11 ครั้งได้กำหนดรูปแบบชุดข้อมูลแต่ละชุด แบ่งออกเป็น 3 รูปแบบ โดยอ้างอิงจากอุณหภูมิที่เซนเซอร์ AMG8833 ตรวจจับได้ในวิดีโอการทดลอง ดังนี้

- รูปแบบที่ 1 หมายถึงช่วงที่ไม่มีการจ่ายกระแสไฟฟ้า
- รูปแบบที่ 2 หมายถึงช่วงที่เริ่มมีการจ่ายกระแสไฟฟ้า
- รูปแบบที่ 3 หมายถึงช่วงที่สายไฟฟ้าเริ่มมีการละลาย

โดยรูปแบบทั้ง 3 จะมีเกณฑ์ที่ใช้ในการกำหนดค่าให้ข้อมูลแต่ชุด ดังนี้

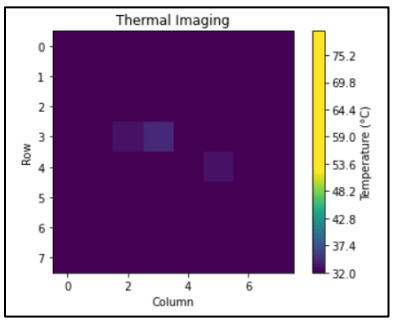
- รูปแบบที่ 1 ค่าอุณหภูมิที่มีค่ามากที่สุดในชุดข้อมูลต้องไม่เกิน 34 องศาเซสเซียส กำหนด status เป็น 0
- รูปแบบที่ 2 ค่าอุณหภูมิที่มีค่ามากที่สุดในชุดข้อมูลมากกว่าเท่ากับ 35 ไม่เกิน 45 องศาเซสเซียส กำหนด status เป็น 1
- รูปแบบที่ 3 ค่าอุณหภูมิที่มีค่ามากที่สุดในชุดข้อมูลมากกว่า 45 องศาเซสเซียส กำหนด status เป็น 2

หลังจากที่กำหนดรูปแบบให้ข้อมูลแต่ละชุด และแต่ละการทดลองจนครบ 11 การทดลอง นำข้อมูลของทุกการทดลองมารวมกันเพื่อใช้ในการสร้างโมเดล ดังรูปที่ 5.28

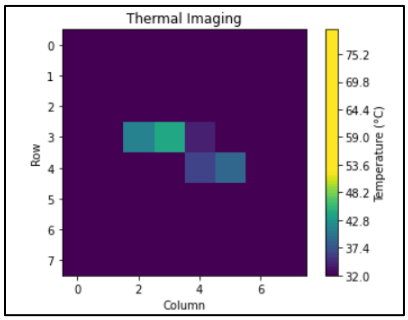
	pixel1	pixel2	pixel3	pixel4	pixel5	pixel6	pixel7	pixel8	pixel9	pixel10	...	pixel56	pixel57	pixel58	pixel59	pixel60	pixel61	pixel62	pixel63	pixel64	status
0	24.50	24.25	25.50	25.00	25.50	25.25	25.75	25.50	24.75	25.25	...	25.25	24.50	23.75	24.50	26.25	27.00	26.25	24.75	25.50	0
1	24.75	25.25	25.50	25.25	25.50	25.25	25.25	26.00	25.00	25.25	...	25.00	24.50	24.25	25.25	26.25	27.00	27.00	25.25	25.25	0
2	24.50	24.75	25.25	25.25	25.50	25.50	25.50	25.25	24.75	25.25	...	25.25	24.75	25.00	24.75	26.25	27.00	26.25	25.00	25.75	0
3	25.50	25.25	25.25	25.25	26.00	25.75	25.75	25.25	25.00	25.75	...	25.50	24.25	24.50	25.25	26.50	27.25	25.00	24.75	25.50	0
4	24.75	25.25	26.00	25.75	25.25	25.50	25.50	25.50	25.00	25.50	...	26.00	24.50	25.00	25.25	27.25	27.25	25.25	24.75	25.50	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
4361	26.00	26.00	26.50	26.00	26.25	26.25	26.00	26.50	26.25	26.25	...	25.25	25.75	25.25	25.75	26.00	26.00	25.00	25.00	25.00	0
4362	25.50	26.25	26.50	26.00	26.00	25.75	25.75	25.75	25.75	26.00	...	24.75	25.25	25.50	26.00	26.25	26.00	25.50	25.00	25.25	0
4363	26.25	26.25	27.00	25.75	26.00	27.00	26.25	26.00	25.75	26.50	...	24.50	25.00	25.75	25.50	25.75	25.75	25.25	25.50	25.25	0
4364	26.25	26.00	27.25	25.75	26.00	26.25	26.00	26.25	26.00	26.00	...	25.75	25.25	25.50	25.50	26.50	26.00	25.00	24.75	25.25	0
4365	25.75	26.00	26.25	26.00	25.75	26.00	26.00	26.50	26.00	26.75	...	25.00	25.00	25.50	25.50	26.25	26.25	25.00	25.50	25.50	0

รูปที่ 5.28 ค่าอุณหภูมิที่ได้จากการทดลองเก็บข้อมูล

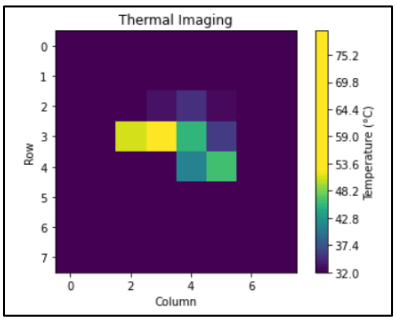
นำข้อมูลในแต่ละรูปแบบมาทำเป็นภาพเพื่อแสดงเห็นถึงจุดความร้อนที่เกิดขึ้น ดังรูปที่ 5.29, 5.30, 5.31 จากรูปจะเห็นความแตกต่างอย่างชัดเจนในความร้อนของสายไฟในแต่ละรูปแบบ โดยจุดที่มีอุณหภูมิสูงจะมีสีเป็นสีเหลือง และจุดที่มีอุณหภูมิต่ำจะเป็นสีน้ำเงินคล้ำ



รูปที่ 5.29 รูปแบบที่ 1



รูปที่ 5.30 รูปแบบที่ 2



รูปที่ 5.31 รูปแบบที่ 3

5.5 การเทรนโมเดลด้วยข้อมูลที่ได้จากการทดลอง

1. ติดตั้งไลบรารีที่จำเป็นในการสร้างโมเดล และ import เพื่อใช้ในการเขียนโค้ด ดังนี้

- pandas เพื่อใช้ในการอ่านไฟล์ข้อมูลอุณหภูมิ
- sklearn เพื่อใช้ในการสร้างโมเดล เทรนโมเดล และทดสอบประสิทธิภาพโมเดล
- joblib เพื่อใช้ในการโหลดโมเดลที่ได้จาก sklearn ดังรูปที่ 5.32

```
import pandas as pd
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
import joblib
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

รูปที่ 5.32 import library

2. อ่านไฟล์ข้อมูลโดยเก็บไว้ที่ตัวแปรชื่อ data ดังรูปที่ 5.33

```
data = pd.read_csv("C:/Users/oatza/Desktop/Project/About/python/cabletrain012.csv")
```

รูปที่ 5.33 read csv

3. แยกข้อมูลออกเป็นข้อมูลทั้งหมดที่ไม่มี status และข้อมูล status เก็บลงตัวแปร X และ Y ตามลำดับ ดังรูปที่ 5.34

```
X = data.drop('status', axis=1)
Y = data['status']
```

รูปที่ 5.34 read csv

4. ทำการแบ่งข้อมูลทั้งหมดที่ไม่มี status (X) ออกเป็นข้อมูลสำหรับการฝึกฝน 80% ข้อมูลสำหรับการทดสอบ 20% และข้อมูล status (Y) ออกเป็นข้อมูลสำหรับการฝึกฝน 80% ข้อมูลสำหรับการทดสอบ 20% จากนั้นเก็บลงในตัวแปร X_train, X_test, Y_train และ Y_test ตามลำดับ ดังรูปที่ 5.35

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=100)
```

รูปที่ 5.35 การแบ่งข้อมูล

5. ทำการฝึกฝนโมเดลโดยกำหนดพารามิเตอร์ จำนวนต้นไม้ (n_estimators) เป็น 100 และ random_state เป็น 100 และทำการฝึกฝนโมเดล โดยใช้ X_train และ Y_train ที่ได้ทำการแยกข้อมูลไว้จากข้อก่อนหน้า ดังรูปที่ 5.36

โดยจำนวนต้นไม้ที่กำหนดเป็น 100 นั้นคือจำนวนต้นไม้ที่ดีที่สุดซึ่งมีผลการประเมินของโมเดลดีที่สุดจะมีการอธิบายต่อในหัวข้อ 6.1.2 การประเมินผลการทดลอง

```
model = RandomForestRegressor(max_depth=None, random_state=100, n_estimators=10)
model.fit(X_train, Y_train)
```

รูปที่ 5.36 การกำหนดพารามิเตอร์

6. ทำการตั้งชื่อไฟล์โมเดล จากนั้นทำการบันทึกโมเดลนำออกมาใช้งาน ดังรูปที่ 5.37

```
model_filename = 'CableModelRandomForestRegressor012.pkl'
joblib.dump(model, model_filename)
```

รูปที่ 5.37 การกำหนดพารามิเตอร์

5.6 การใช้งาน ESP32-CAM

การใช้งานเริ่มแรกจะต้องทำกำหนดอินเทอร์เน็ตที่ต้องการเชื่อมต่อ กำหนด URL ของ Firebase Realtime Database และกำหนดคีย์การตรวจสอบของ Firebase Auth ของ Firebase ดังรูปที่ 5.38

```
#include <WiFi.h>
#include <FirebaseESP32.h>
#include "esp_camera.h"
#include "base64.h"


//const char* WIFI_SSID = "true_home2G_A08";
//const char* WIFI_PASSWORD = "432fmckr";

const char* WIFI_SSID = "sawedus";
const char* WIFI_PASSWORD = "supiitchas";


#define FIREBASE_HOST "https://dbpj-2-default-rtdb.firebaseio.com/"
#define FIREBASE_AUTH "uo3e4LYknRUBoTu5glUzWsNXCKsnbviLGAndNmbT"
```

รูปที่ 5.38 การกำหนดอินเทอร์เน็ตที่เชื่อม กำหนด URL และกำหนดคีย์ของ Firebase

```
84  void loop() {
85    // Take a photo
86    camera_fb_t* fb = esp_camera_fb_get();
87
88    if (!fb) {
89      Serial.println("Failed to capture image");
90      return;
91    }
92
93      if (Firebase.setBlob(firebaseData, "/deviceID/001/image", fb->buf, fb->len)) {
94
95      Serial.println("Photo uploaded successfully");
96    } else {
97      Serial.println("Failed to upload photo");
98    }
99
100      esp_camera_fb_return(fb);
101
102    delay(3000);
103
104 }
```

รูปที่ 5.39 การกำหนด path และ กำหนดเวลาในการจับภาพ

จากรูปที่ 5.39 กำหนด path ของ Firebase อยู่ที่ /deviceID/001/image หมายความคือ ภาพที่ได้จากอุปกรณ์ที่ 001 จะถูกจัดเก็บไว้อยู่ที่ path นี้ และกำหนดเวลาในการจับภาพทุก 3 วินาที ซึ่งหลังจากนั้นจะนำ base64 มาแสดงเป็นภาพบนเว็บไซต์ของผู้ใช้เมื่อมีเหตุเกิดภาพที่ได้จะเป็นภาพรูปแบบ base64 ดังรูปที่ 5.40 และ 5.41 ตามลำดับ


```
https://dbpj-2-default-rtdb.firebaseio.com/
  deviceID
    001
      UserID: "U42b48901c79cb57f471cffba749f690a"
      image: "blob,base64,/9j/4AAQSkZJRgABAQEAAAAAAAD/2wBDAAoHCAkIBgoJCAkLCwoMDxkQDw4ODx8WFxIZJCAmJiQgIyIoLToxKCs2KyIjMkQzNjs
```


รูปที่ 5.40 ภาพรูปแบบ base64

รูปที่ 5.41 ภาพที่ได้จากนำ base64 มาแสดง

บทที่ 6 ผลการดำเนินงาน ข้อจำกัด และข้อเสนอแนะ

บทนี้อธิบายถึงความก้าวหน้าผลการดำเนินงาน ข้อจำกัด และแนวทางการแก้ไขปัญหาจากการทำโครงงานปริญญานิพนธ์ในช่วงเวลาที่ผ่านมา

6.1 ผลการดำเนินงาน

6.1.1 การทดสอบประสิทธิภาพของการทำนาย

แบ่งชุดข้อมูลที่ใช้ในการทดสอบเป็น 3 รูปแบบ คือ รูปแบบที่ 1 ค่าอุณหภูมิที่มีค่ามากที่สุดในชุดข้อมูลต้องไม่เกิน 34 องศาเซสเซียส (0) รูปแบบที่ 2 ค่าอุณหภูมิที่มีค่ามากที่สุดในชุดข้อมูลมากกว่าเท่ากับ 35 ไม่เกิน 45 องศาเซสเซียส (1) รูปแบบที่ 3 ค่าอุณหภูมิที่มีค่ามากที่สุดในชุดข้อมูลมากกว่า 45 องศาเซสเซียส (2) ใช้ฝึกฝน 80 และทดลอง 20 โดยใช้ค่ามาตราวัดที่ใช้กับ Regressor อ้างอิงจากเว็บไซต์ของ Scikit-learn [15] เพื่อดูความคลาดเคลื่อนของโมเดล และการอธิบายข้อมูลของโมเดล มีดังนี้

1) Mean Absolute Error (MAE): ค่านี้วัดความคลาดเคลื่อนเฉลี่ยระหว่างค่าทำนาย (predicted values) กับค่าจริง (actual values) โดยไม่สนใจทิศทางของความคลาดเคลื่อน ค่า MAE ต่ำมากๆ หมายถึงโมเดลมีความแม่นยำในการทำนายสูง ค่า MAE = 0 หมายถึงการทำนายที่ถูกต้องแม่นยำที่สุด โดยสูตรที่ใช้ในการคำนวณคือ

$$\mathrm{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$$

กำหนดให้ N คือ จำนวนข้อมูลทั้งหมด

y_i คือ ค่าความจริง

$\hat{y}_i$ คือ ค่าการทำนาย

2) Mean Squared Error (MSE): ค่านี้วัดความคลาดเคลื่อนระหว่างค่าทำนายและค่าจริงโดยรวมโดยให้ความสำคัญกับความแตกต่างในระดับแต่ละตัวแปร ค่า MSE นี้จะมีค่ามากกว่า MAE เนื่องจากมันยกกำลังสองค่าความคลาดเคลื่อน ค่า MSE ก็คือความแตกต่างระหว่างค่าที่ทำนายและค่าจริงยกกำลังสองแล้วนำมาหาค่าเฉลี่ยโดยสูตรที่ใช้ในการคำนวณ คือ

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

กำหนดให้ N คือ จำนวนข้อมูลทั้งหมด

y_i คือ ค่าความจริง

$\hat{y}_i$ คือ ค่าการทำนาย

3) Root Mean Squared Error (RMSE): ค่า RMSE คือรากที่สองของค่า MSE ค่านี้ใช้วัดความคลาดเคลื่อนเช่นเดียวกับ MSE แต่มันให้ค่าที่อ่านง่ายกว่าเนื่องจากมีหน่วยวัดเดียวกับค่าที่ทำนายและค่าจริง สูตรที่ใช้คำนวณ คือ

$$\mathrm{RMSE} = \sqrt{MSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}$$

กำหนดให้ N คือ จำนวนข้อมูลทั้งหมด

y_i คือ ค่าความจริง

$\hat{y}_i$ คือ ค่าการทำนาย

4) R-squared (R^2): ค่า R^2 มักถูกใช้เป็นวิธีในการวัดความแม่นยำของโมเดลสำหรับการทำนายค่าตัวเลข ค่า R^2 อยู่ในช่วง 0 ถึง 1 โดยค่า R^2 ที่ใกล้ 1 หมายถึงโมเดลสามารถอธิบายความคลาดเคลื่อนของข้อมูลได้ดี และค่า R^2 ที่ใกล้ 0 หมายถึงโมเดลไม่สามารถอธิบายความคลาดเคลื่อนของข้อมูลได้ดี

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y}_i)^2}$$

กำหนดให้ $\hat{y}_i$ คือ ค่าการทำนาย

$\bar{y}_i$ คือ ค่าเฉลี่ยของข้อมูลต้นฉบับ

6.1.2 การประเมิณผลการทดลอง

ในการสร้างโมเดล Random forest ต้องทำการกำหนดจำนวนต้นไม้ ซึ่งจะมีผลต่อประสิทธิภาพในการทำนายผล และเวลาที่ใช้ในการทำนายผล โดยได้ทำการสร้างโมเดลขึ้นมาตามขั้นตอนดังหัวข้อที่ 5.5 จากนั้น ได้นำโมเดลที่ได้ไปทดลองด้วยข้อมูล X_test ที่ถูกแบ่งเอาไว้ จะได้ผลลัพธ์จากการทำนายเก็บลงตัวแปร Y_pred ในขั้นตอนถัดไป จะนำตัวแปร Y_pred และ Y_test มีความหมายคือ ผลลัพธ์ที่ได้จากการทำนาย และผลลัพธ์จริงตามลำดับ ไปใช้ในการคำนวนในการประเมิณโมเดลต่อไปดังรูปที่ 6.1

```
Y_pred = model.predict(X_test)

mae = mean_absolute_error(Y_test, Y_pred)
print(f'MAE: {mae}')

mse = mean_squared_error(Y_test, Y_pred)
print(f'MSE: {mse}')

rmse = mean_squared_error(Y_test, Y_pred, squared=False)
print(f'RMSE: {rmse}')

r2 = r2_score(Y_test, Y_pred)
print(f'R2: {r2}')
```

รูปที่ 6.1 โค้ดการประเมิณโมเดล

ทำการทำซ้ำในขั้นตอนทั้งหมดใหม่โดยปรับจำนวนต้นไม้ จะได้ผลการประเมินโมเดลที่ได้จากการปรับจำนวนต้นไม้ที่ใช้ในการสร้างโมเดล และเปรียบเทียบค่า MAE, ค่า MSE, ค่า RMSE และค่า R^2 เพื่อหาจำนวนต้นไม้ที่เหมาะสมในการนำมาใช้งานทำนายผล ดังนี้

กำหนดจำนวนต้นไม้ 50 ต้น

```
MAE: 0.010915331807780324
MSE: 0.0035336384439359266
RMSE: 0.059444414741302035
R2: 0.9947914131288436
```

รูปที่ 6.2 ผลการทดลองโมเดลจำนวนต้นไม้ 50 ต้น

กำหนดจำนวนต้นไม้ 100 ต้น

```
MAE: 0.011567505720823802
MSE: 0.003517505720823799
RMSE: 0.05930856363817791
R2: 0.9948151927800816
```

รูปที่ 6.3 ผลการทดลองโมเดลจำนวนต้นไม้ 100 ต้น

กำหนดจำนวนต้นไม้ 150 ต้น

```
MAE: 0.011853546910755148
MSE: 0.0036510551741673024
RMSE: 0.060423961920477394
R2: 0.9946183407420558
```

รูปที่ 6.4 ผลการทดลองโมเดลจำนวนต้นไม้ 150 ต้น

กำหนดจำนวนต้นไม้ 200 ต้น

```
MAE: 0.011721967963386721
MSE: 0.0035681636155606407
RMSE: 0.05973410764011329
R2: 0.9947405229886943
```

รูปที่ 6.5 ผลการทดลองโมเดลจำนวนต้นไม้ 200 ต้น

ตารางที่ 6.1 ตารางเปรียบเทียบผลของจำนวนต้นไม้

จำนวนต้นไม้	MAE	MSE	RMSE	R2	ผลรวม
50	4	3	3	3	13
100	3	4	4	4	15
150	1	1	1	1	4
200	2	2	2	2	8

จากตารางที่ 6.1 จะมีการให้คะแนนในค่า MAE, MSE, RMSE และ R2 โดยการเปรียบเทียบค่า MAE, MSE และ RMSE ถ้าผลลัพธ์ที่ออกมามีค่าเข้าใกล้ 0 จะแสดงถึงความแม่นยำของโมเดลสูง และค่า R^2 ถ้ามีค่าเข้าใกล้ 1 จะหมายถึงโมเดลสามารถอธิบายความคลาดเคลื่อนของข้อมูลได้ดี การให้คะแนนจะให้จากผลลัพธ์ที่ได้ในแต่ละค่าเทียบกับจำนวนต้นไม้อื่น โดยผลลัพธ์ที่ดีที่สุดจะให้เป็น 4 และลดลงมาจนถึง 1 จากนั้นรวมผลลัพธ์โดยการนำคะแนนทุกค่ามารวมกันเพื่อหาจำนวนต้นไม้ที่มีคะแนนเยอะที่สุด และนำมาใช้งาน

หลังจากที่ทำการปรับจำนวนต้นไม้ และเปรียบเทียบค่า จำนวนต้นไม้ที่ดีที่สุด คือ 100 ต้น มีผลลัพธ์ดังรูปที่ 6.2

6.2 ข้อจำกัด

1) การเชื่อมต่ออินเทอร์เน็ตของอุปกรณ์ต้องเชื่อมต่อเข้ากับ Pocket Wi-fi ที่มาพร้อมกับอุปกรณ์เท่านั้น
2) อุปกรณ์จำเป็นต้องต่อเข้ากับแบตสำรองเพื่อเริ่มใช้งาน
3) ระยะที่ติดตั้งอุปกรณ์ไม่เกิน 30 เซนติเมตร
4) จุดที่ติดตั้งอุปกรณ์ควรเป็นจุดที่เสียบสายไฟไว้ตลอดเวลา

6.3 ข้อเสนอแนะ

1) ควรเพิ่มอุปกรณ์ Relay ไว้สำหรับตัดกระแสไฟหลังได้รับการแจ้งเตือน

บรรณานุกรม

[1] ก. ใ. พ. ตั้งตราชู, “ระบบคัดกรองผู้ที่มีอาการป่วยในที่สาธารณะ,” *วารสารวิทยาศาสตร์ เทคโนโลยีและนวัตกรรม,* 2020.

[2] จ. วารินทร์, ต่อยอดพัฒนาเว็บแอพพลิเคชันด้วย React + Firebase, กรุงเทพฯ: รีไวว่า, 2565.

[3] ชาญณรงค์ สมเพ็ชร,พชร ทองฉิม,ลาภิน ปานสิลา,นครินทร์ สุขเพ็ง และเตชสิทธิ์ ใจคำ, “เครื่องแจ้งเตือนและควบคุมอัคคีภัยผ่าน Line Notify,” 2018. [ออนไลน์]. Available: https://elecschool.navy.mi.th/pro/doc61/09.pdf. [%1 ที่เข้าถึง5 December 2022].

[4] ธีรถวัลย์ ปานกลาง, รักเกียรติ ขำดำรงเกียรติ, และ รัตนสุดา สุภดนัยสร, “การพัฒนาระบบตรวจวัดอุณหภูมิร่างกายแจ้งเตือนข้อความและภาพถ่ายผ่านแอพพลิเคชั่นไลน์,” *วารสารวิชาการคณะเทคโนโลยีอุตสาหกรรม,* %1Vol 16 No 1 (2564): เทพสตรี I-TECH, 2564.

[5] H. Team, “HIVEMQ,” 31 พฤษภาคม 2023. [ออนไลน์]. Available: https://www.hivemq.com/article/how-to-get-started-with-mqtt/.

[6] L. Developers. [ออนไลน์]. Available: https://developers.line.biz/en/docs/liff/using-user-profile/#use-user-info-on-server.

[7] n. Team, “nodejs,” [ออนไลน์]. Available: https://nodejs.dev/en/learn/.

[8] S. Team, “Scikit-learn,” [ออนไลน์]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html.

[9] S. Team, “Scikit-learn,” [ออนไลน์]. Available: https://scikit-learn.org/stable/modules/model_evaluation.html#.

[10] 18 November 2021. [ออนไลน์]. Available: https://www.ai-corporation.net/2021/11/18/what-is-arduino-ide/#1648794579978-4ac5b2e6-420a. [%1 ที่เข้าถึง13 Decemver 2022].

[11] “Adafruit,” 28 jun 2017. [ออนไลน์]. Available: https://learn.adafruit.com/adafruit-amg8833-8x8-thermal-camera-sensor/overview.

[12] “digitalarea,” [ออนไลน์]. Available: https://digitalarea.co/line-notify-beginner/.

[13] “designil,” [ออนไลน์]. Available: https://www.designil.com/react-%E0%B8%84%E0%B8%B7%E0%B8%AD%E0%B8%AD%E0%B8%B0%E0%B9%84%E0%B8%A3/.

[14] “ayarafun,” 13 August 2015. [ออนไลน์]. Available: http://www.ayarafun.com/2015/08/introduction-arduino-esp8266-nodemcu/.

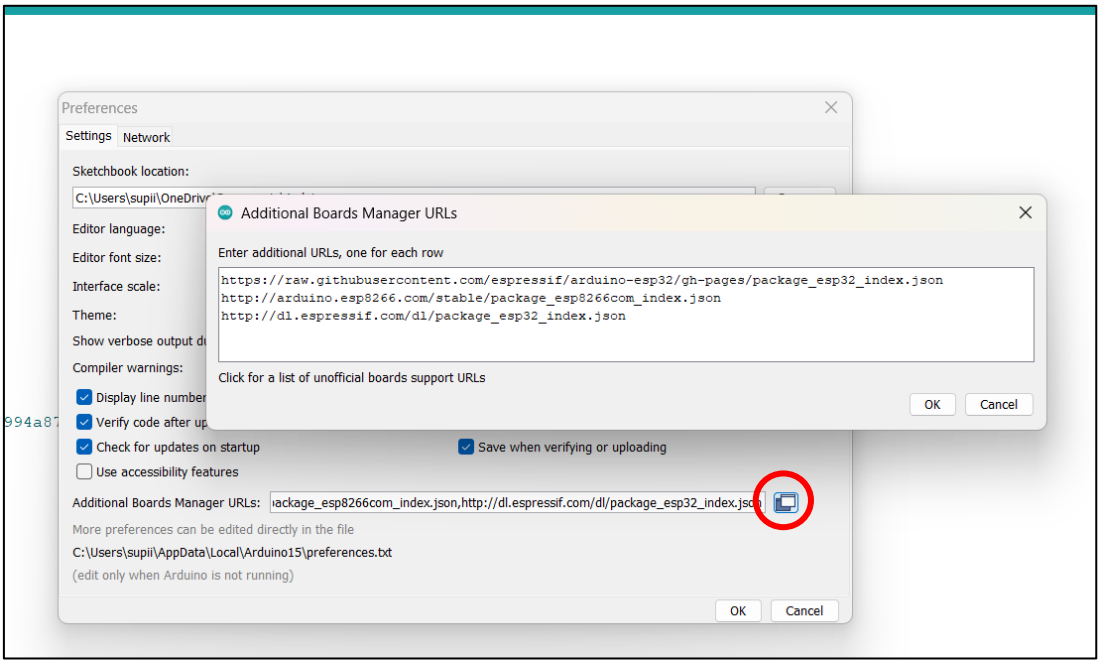
[15] “hwlibre,” 22 February 2021. [ออนไลน์]. Available: https://www.hwlibre.com/th/esp32-cam/?msclkid=bba9c3f5bd6711ecb2220db86c70035b.

ภาคผนวก ก คู่มือการติดตั้งระบบ

ก.1 การเซ็ทระบบ Arduino IDE

ก.1.1 การเพิ่มโมดูล

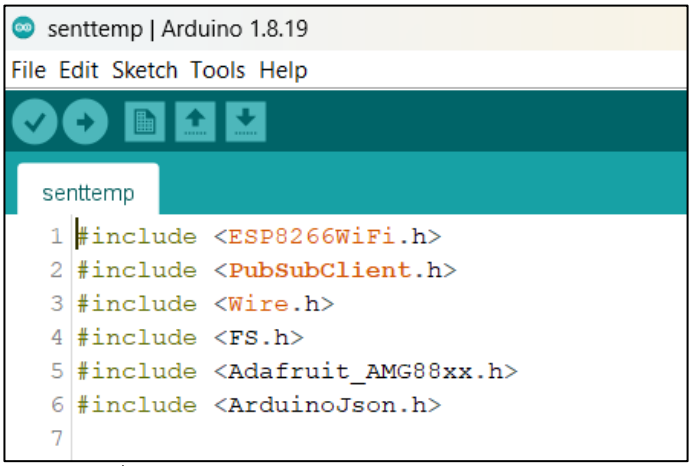
เมื่อติดตั้งโปรแกรม Arduino IDE สำเร็จจะต้องทำการเพิ่มโมดูลก่อน โดยกดที่ไป File > Preferences หลังจากนั้นไปที่ Additional Boards Manager URLs ทำการเพิ่ม URL ดังรูปผนวกที่ ก.1.1



รูปผนวกที่ ก.1.1 การเพิ่มโมดูลในโปรแกรม Arduino IDE

ก.1.2 ไลบรารีโมดูล ESP8266 กับ AMG8833

ติดตั้งไลบรารีดังรูปผนวกที่ ก.1.2



รูปผนวกที่ ก.1.2 ไลบรารีโมดูล ESP8266 กับ AMG8833

ก.1.3 ไลบรารีโมดูล ESP32-CAM

ติดตั้งไลบรารีดังรูปผนวกที่ ก.1.3

รูปผนวกที่ ก.1.3 ไลบรารีโมดูล ESP32-CAM

ก.2 การติดตั้งไลบรารีที่ใช้ใน vs code

ก.2.1 ไลบรารีเซิร์ฟเวอร์

ไลบรารีที่ต้องติดตั้งมีดังรูปผนวกที่ ก.2.1

```
"dependencies": {
  "axios": "^1.4.0",
  "body-parser": "^1.20.2",
  "cors": "^2.8.5",
  "dotenv": "^16.3.1",
  "express": "^4.18.2",
  "firebase": "^9.22.1",
  "firebase-admin": "^11.9.0",
  "helmet": "^7.0.0",
  "jsonwebtoken": "^9.0.0",
  "moment": "^2.29.4",
  "moment-timezone": "^0.5.43",
  "mqtt": "^4.3.7",
  "python-shell": "^5.0.0"
}
```

รูปผนวกที่ ก.2.1 ไลบรารีเซิร์ฟเวอร์

ก.2.2 ไลบรารีเว็บผู้ใช้

ไลบรารีที่ต้องติดตั้งมีดังรูปผนวกที่ ก.2.2

```
"dependencies": {
  "@emotion/react": "^11.11.1",
  "@emotion/styled": "^11.11.0",
  "@line/liff": "^2.22.2",
  "@mui/icons-material": "^5.11.16",
  "@mui/joy": "^5.0.0-beta.3",
  "@mui/material": "^5.13.6",
  "@mui/x-data-grid-generator": "^6.9.1",
  "@mui/x-data-grid-pro": "^6.9.1",
  "@testing-library/jest-dom": "^5.16.5",
  "@testing-library/react": "^13.4.0",
  "@testing-library/user-event": "^13.5.0",
  "axios": "^1.4.0",
  "react": "^18.2.0",
  "react-dom": "^18.2.0",
  "react-router-dom": "^6.14.0",
  "react-scripts": "5.0.1",
  "web-vitals": "^2.1.4"
},
```

รูปผนวกที่ ก.2.2 ไลบรารีเว็บผู้ใช้

ก.2.3 ไลบรารีเว็บแอดมิน

ไลบรารีที่ต้องติดตั้งมีดังรูปผนวกที่ ก.2.3

```
"dependencies": {
  "@emotion/react": "^11.11.1",
  "@emotion/styled": "^11.11.0",
  "@mui/icons-material": "^5.14.3",
  "@mui/material": "^5.14.5",
  "@testing-library/jest-dom": "^5.17.0",
  "@testing-library/react": "^13.4.0",
  "@testing-library/user-event": "^13.5.0",
  "axios": "^1.4.0",
  "firebase": "^10.2.0",
  "react": "^18.2.0",
  "react-dom": "^18.2.0",
  "react-router-dom": "^6.15.0",
  "react-scripts": "5.0.1",
  "web-vitals": "^2.1.4"
},
```

รูปผนวกที่ ก.2.3 ไลบรารีเว็บแอดมิน

ก.2.4 ไลบรารีสร้างโมเดล

ไลบรารีที่ต้องติดตั้งมีดังรูปผนวกที่ ก.2.4

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
import joblib
from sklearn.metrics import accuracy_score, confusion_matrix
```

รูปผนวกที่ ก.2.4 ไลบรารีสร้างโมเดล

ก.2.5 ไลบรารี API Model

ไลบรารีที่ต้องติดตั้งมีดังรูปผนวกที่ ก.2.5

```
from flask import Flask, request, jsonify
from flask_cors import CORS
import joblib
import pandas as pd
```

รูปผนวกที่ ก.2.5 ไลบรารี API Model

ภาคผนวก ข การทำงานส่วนของ Server

การทำงานในส่วน Server เพื่อติดต่อกับส่วนต่าง ๆ ของ ของเว็บและ MQTT โดยมีการทำงานในส่วนต่าง ๆ ดังนี้

ข.1 การทำงานของ Server ส่วนการติดต่อกับ MQTT

เริ่มจากนำเข้าไลบรารี MQTT จากนั่นกำหนดค่าที่ใช้ในการเชื่อมต่อกับ MQTT Broker ลงในตัวแปร optionsMqtt โดยจะ กำหนดค่า ดังนี้

1. ค่า host จะใช้เป็นที่อยู่ในการติดต่อกับ MQTT Broker โดยสร้าง cluster ที่เว็ปไซต์ของ HiveMQ Cloud และนำ Cluster URL ที่ได้มากำหนดลงตัวแปร host

2. ค่า port สำหรับ MQTT over TLS (mqtts) จะใช้ port 8883

3. ค่า protocol ระบุเป็น mqtts คือ MQTT over TLS ซึ่งเป็นการเชื่อมต่อ MQTT ที่มีการเข้ารหัสและรักษาความปลอดภัย

4. ค่า username และ password ใช้สำหรับการยืนยันตัวตนเข้าสู่ระบบกับ MQTT Broker โดยจะต้องทำการสร้างขึ้นที่เว็ปไซต์ HiveMQ Cloud และนำในกำหนดลงตัวแปร

จากนั้นเรียกใช้คำสั่ง ‘mqtt.connect(optionsMqtt)’ เพื่อทำการเชื่อมต่อกับ MQTT Broker โดยใช้ค่าที่กำหนดไว้ใน optionsMqtt ดังรูปผนวกที่ ข.1.1

```
const mqtt = require('mqtt')

const optionsMqtt = {
    host: 'f8cdbde810784542a9994a87cd551d08.s2.eu.hivemq.cloud',
    port: 8883,
    protocol: 'mqtts',
    username: 'Servertest1',
    password: 'Servertest1'
}
const client = mqtt.connect(optionsMqtt)
```

รูปผนวกที่ ข.1.1 การเชื่อมต่อกับ MQTT Broker

เมื่อเชื่อมต่อกับ MQTT Broker แล้วคำสั่ง ‘client.on('connect', () => { ... })’ จะทำงาน และเรียกใช้ฟังก์ชัน getAllDevice เพื่อทำการอ่านข้อมูลจาก Realtime Database โดยจะอ่านข้อมูลของอุปกรณ์ทำหมดที่มีผู้ใช้ใช้งานอยู่ และทำการสมัครรับข้อมูลไปยังหัวข้อของ MQTT โดยใช้ deviceID ที่ได้เป็นหัวข้อที่สมัครรับข้อมูล ดังรูปผนวกที่ ข.1.2

```
client.on('connect', () => {
    getAllDevice()
})

const getAllDevice = async () => {
    try {
        const deviceIDSanpshot = await realtimeDB.ref('deviceID').once('value')
        const data = deviceIDSanpshot.val()
        for (const deviceID in data) {
            if (data[deviceID].UserID != 'none') {
                client.subscribe(deviceID)
                console.log(`Subscribe to the deviceID "${deviceID}"`)
            }
        }
    }
    catch (error) {
        console.error(error)
    }
}
```

รูปผนวกที่ ข.1.2 สมัครรับข้อมูล

เมื่อสมัครรับข้อมูลแล้ว คำสั่ง client.on('message', () => { ... }) จะทำงานเมื่อมีข้อมูลส่งมาจากหัวข้อที่ได้ทำการสมัคร รับ ซึ่งข้อมูลที่ถูกส่งเข้ามาจะถูกแบ่งส่งเป็น 2 ชุด โดยชุดแรก จะมี '[' อยู่ในข้อมูล และชุดที่ 2 จะมี ']' อยู่ในข้อมูล และ ข้อมูลยังไม่ถูกทำให้เป็นทศนิยม ตัวแปร topicData คืออ็อบเจ็กต์ที่ใช้ในการเก็บข้อมูลที่เกี่ยวข้องกับหัวข้อ MQTT เมื่อข้อมูล ถูกส่งเข้ามาจะทำการตรวจสอบชื่อของหัวข้อจากใน topicData หากว่าไม่มีจะทำการสร้างขึ้นมาเพื่อรอเก็บข้อมูลที่เข้ามา จากนั้นตรวจสอบว่าข้อมูลเป็นข้อมูลชุดแรก หรือชุดที่ 2 หากเป็นข้อมูลชุดแรกจะทำการเก็บข้อมูลไปยังหัวข้อที่ได้รับข้อมูลมา ใน topicData และรอรับข้อมูลชุดที่ 2 และหากเป็นข้อมูลชุดที่ 2 จะนำข้อมูลชุดแรกในหัวข้อที่ตรงกันมาทำการต่อเข้า ด้วยกันโดยขั้นด้วย ',' และลบข้อมุลชุดแรกออกเพื่อเตรียมรับข้อมูลชุดใหม่ ดังรูปผนวกที่ ข.3 จากนั้นนำข้อมูล 2 ชุดที่ต่อกันแล้วมาแต่ยังไม่เป็นทศนิยม มาหาร 100 เพื่อให้ข้อมูลอยู่ในรูปแบบทศนิยม และส่งข้อมูลไปใช้ในการทำนายผลที่ฟังก์ชัน PostApiModel ดังรูปผนวกที่ ข.4 หากว่าผลลัพธ์จากการทำนายมากกว่า 1 จะทำการค้นหา Token LINE Notify ของผู้ใช้จากหัวข้อที่ได้รับข้อมูลในฟังก์ชัน findTokenByDeviceID ดังรูปผนวกที่ ข.5 และทำการนำภาพที่ได้จาก Realtime Database ไปบันทึกยัง Firestore Database จากหัวข้อที่ได้รับข้อมูลในฟังก์ชัน realtimeToFirestore ดังรูปผนวกที่ ข.7 และทำการส่งแจ้งเตือนไลน์ไปยังผู้ใช้โดยใช้ Token LINE Notify ที่ได้ค้นหามาในฟังก์ชัน sendLineNotify ดังรูปผนวกที่ ข.6

```
let topicData = {};

client.on('message', async (topic, message) => {
    try {
        if (!(topic in topicData)) {
            topicData[topic] = '';
        }
        const receivedData = message.toString();
        if (receivedData.includes('[')) {
            topicData[topic] = receivedData;
            console.log('รับข้อมูลส่วนที่ 1')
        } else {
            if (topicData[topic] && receivedData.includes(']')) {
                console.log('รับข้อมูลส่วนที่ 2')
                const combinedData = topicData[topic] + ',' + receivedData
                topicData[topic] = '';
                const numbers = combinedData
                    .substring(1, combinedData.length - 1)
                    .split(',')
                    .map(item => (parseFloat(item) / 100).toFixed(2));

                const data = "[" + numbers.toString() + "]"
                console.log(`Divided data for ${topic}:`, data);
                const status = await PostApiModel(data)
                console.log(`ผลลัพธ์ = ${status}`)
                if (status > 1) {
                    const tokenline = await findTokenByDeviceID(topic)
                    await realtimeToFirestore(topic)
                    for (const token of tokenline) {
                        sendLineNotify(token, data.toString(), topic)
                    }
                }
            } else {
                console.log(`รับข้อมูลไม่ครบ`);
            }
        }
    }
    catch (error) {
        console.error(error)
    }
})
```

รูปผนวกที่ ข.1.3 รับข้อมูลจากหัวข้อที่สมัครรับ

```
const PostApiModel = async (data) => {
    try {
        console.log('ส่งข้อมูลไปทำนาย...')
        const json = JSON.parse(data);
        const dataFormatted = json.reduce((acc, pixel, index) => {
            acc[`pixel${index + 1}`] = pixel
            return acc;
        }, {});
        const response = await axios.post('http://127.0.0.1:3000/predict', dataFormatted)
        return response.data.prediction
    }
    catch (err) {
        console.error(err)
    }
}
```

รูปผนวกที่ ข.1.4 ส่งข้อมูลไปยัง API model

```
const findTokenByDeviceID = async (deviceID) => {
    try {
        const tokenLines = []
        const DeviceSnapshot = await firestoreDB
            .collectionGroup('rooms')
            .where('deviceID', '==', deviceID)
            .get()

        if (DeviceSnapshot.empty) {
            console.log(`No users have subscribed to the deviceID:${deviceID}`)
            return
        }

        for (const doc of DeviceSnapshot.docs) {
            const docData = doc.ref.parent.parent.id
            const userDoc = await firestoreDB
                .collection('Users')
                .doc(docData)
                .get()

            if (userDoc.exists) {
                const tokenline = userDoc.data().tokenLine
                tokenLines.push(tokenline)
            } else {
                console.log(`User document not found for ID: ${docData}`)
            }
        }

        return tokenLines
    } catch (err) {
        console.error(err)
    }
}
```

รูปผนวกที่ ข.1.5 ค้นหา Token LINE Notify

```
const sendLineNotify = async (token, topic) => {
    try {
        const DeviceSnapshot = await firestoreDB
            .collectionGroup('rooms')
            .where('deviceID', '==', topic)
            .get()

        if (DeviceSnapshot.empty) {
            console.log(`No Device found with deviceID: ${topic}`)
            return
        }

        const deviceDoc = DeviceSnapshot.docs[0]
        const deviceName = deviceDoc.data().deviceName

        const response = await axios({
            method: 'post',
            url: 'https://notify-api.line.me/api/notify',
            headers: {
                'Content-Type': 'application/x-www-form-urlencoded',
                'Authorization': `Bearer ${token}`
            },

            data: `message=${deviceName} พบเหตุการณ์ผิดปกติ โปรดตรวจสอบรูปที่ https://thermal-notify.netlify.app/login`
        })
        console.log('Line Notify message sent:', response.data)
    } catch (error) {
        console.error('Error sending Line Notify message:', error.response.data.message)
    }
}
```

รูปผนวกที่ ข.1.6 ส่งแจ้งเตือนไปยัง LINE

```
const realtimeToFirestore = async (deviceID) => {
    try {
        const imageRef = realtimeDB.ref(`deviceID/${deviceID}/image`)
        const snapshot = await imageRef.once('value')
        const imageData = snapshot.val()
        const imageSplit = imageData.split('blob,')[1]
        const image = `data:image/jpeg;${imageSplit}`
        // console.log(image);
        const DeviceSnapshot = await firestoreDB
            .collectionGroup('rooms')
            .where('deviceID', '==', deviceID)
            .get()
        if (!DeviceSnapshot.empty) {
            const deviceDoc = DeviceSnapshot.docs[0].id
            const userDoc = DeviceSnapshot.docs[0].ref.parent.parent.id
            const currentDate = new Date()
            const thaiTimeZone = 'Asia/Bangkok'
            const localTimeString = moment.tz(currentDate, thaiTimeZone).format('YYYY-MM-DD HH:mm:ss')
            const date = localTimeString.split(' ')[0]
            const time = localTimeString.split(' ')[1]
            const HistoryDocRef = firestoreDB
                .collection('Users')
                .doc(userDoc)
                .collection('rooms')
                .doc(deviceDoc)
                .collection('History')
                .doc(date);

            await HistoryDocRef.set({
                date: date
            });

            await HistoryDocRef.collection('Image').doc(time).set({
                time: time,
                image: image
            });
        }
    }
    catch (error) {
        console.error('Error reading data from Realtime Database:', error)
    }
}
```

รูปผนวกที่ ข.1.7 บันทึกข้อมูลไปยัง Firestore Database

ข.2 การทำงานของ Server ส่วนการติดต่อกับ API Model

สร้าง API ที่มีการเรียกใช้โมเดล CableModelRandomForestRegressor012.pkl เพื่อให้ Server ส่งข้อมูลเข้ามาเพื่อใช้ในการทำนายผล โดยจะรับบข้อมูลที่ถูกส่งมาจาก Server และแปลงเป็น DataFrame จากนั้นนำค่าที่ได้ไปทำนายผลและส่งผลลัพธ์กลับไปยัง Server ดังรูปผนวกที่ ข.8

```
from flask import Flask, request, jsonify
from flask_cors import CORS
import joblib
import pandas as pd

app = Flask(__name__)
CORS(app)

with open('C:/Users/oatza/Desktop/Project/pj-thermal-model-randomforest/CableModelRandomForestRegressor012.pkl', 'rb') as file:
    model = joblib.load(file)
    print(type(model))

@app.route('/predict', methods=['POST'])
def predict():
    data = request.get_json()
    new_data = pd.DataFrame([data])
    prediction = model.predict(new_data)
    return jsonify({'prediction': prediction[0]})


if __name__ == '__main__':
    app.run(port=3000)
```

รูปผนวกที่ ข.2.1 API Model